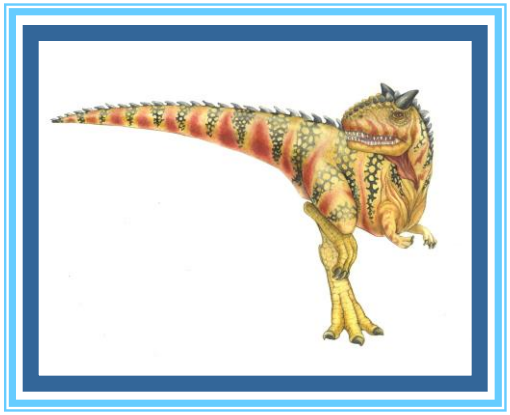
Chapter 4: Threads & Concurrency



Silberschatz, Galvin and Gagne ©2018



Outline

- Overview
- Multicore Programming
- Multithreading Models
- Thread Libraries
- Implicit Threading
- Threading Issues
- Operating System Examples





Objectives

- Identify the basic components of a thread, and contrast threads and processes
- Describe the benefits and challenges of designng multithreaded applications
- Illustrate different approaches to implicit threading including thread pools, fork-join, and Grand Central Dispatch
- Describe how the Windows and Linux operating systems represent threads
- Design multithreaded applications using the Pthreads, Java, and Windows threading APIs



MOTIVATION: WHY MULTITHREADING?

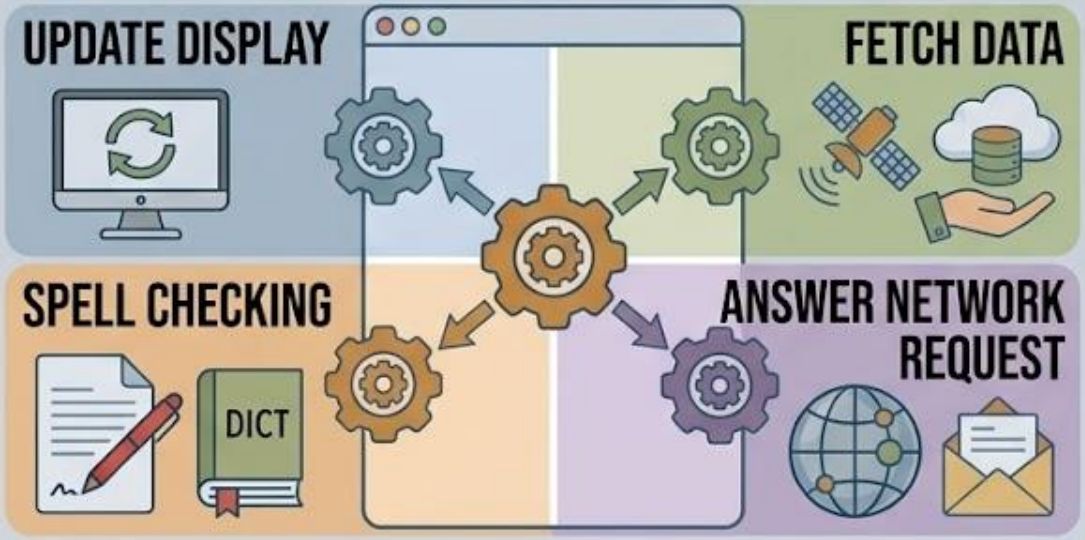
MODERN APPLICATIONS & TASKS

**MOST APPLICATIONS ARE MULTITHREADED



Threads run within a single application address space.

**MULTIPLE TASKS VIA SEPARATE THREADS



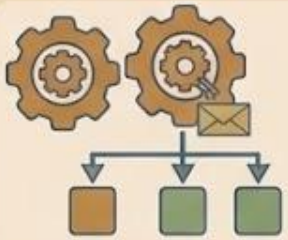
KEY BENEFITS

- **SIMPLIFY CODE, INCREASE EFFICIENCY**

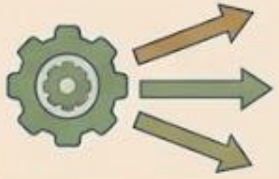


Modular design, parallel processing, direct resource access (e.g., in bounded capacity queues like image_13.png).

- **LIGHT-WEIGHT THREAD CREATION vs HEAVY-WEIGHT PROCESS CREATION**



Process creation is **heavy-weight**
High overhead, separate memory and resources.



Thread creation is **light-weight**
Low overhead, shared memory and resources.

Kernels are generally multithreaded: essential for managing resources, scheduling, and system operations.

THREAD CONCEPTS: BASIC UNIT & PROCESS STRUCTURE

THREAD: BASIC UNIT OF CPU UTILIZATION

A thread is a **basic unit of CPU utilization**; it comprises a thread ID, a program counter (PC), a register set, and a stack.

ADAPTER

- **Thread ID** per thread.
- **Program Counter (PC)**
- **Register Set**
- **Stack**
- **Stack**

Shares with other threads...
code, data, resources

**TRADITIONAL SINGLE-THREADED PROCESS

ONE

ONE threadis (interenpees). A traditional process a single thread of control.

Process

- Code Section
- Data Section
- OS Resources (Files, Signals)

ONE

**MULTITHREADED PROCESS

If a process has multiple threads of

Process

MULTIPLE (e.g., three) thread gears

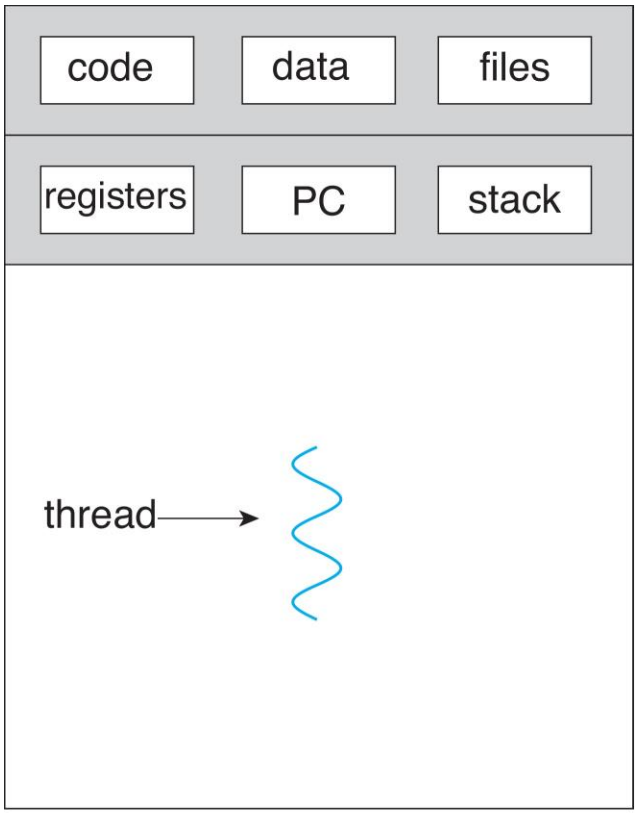
T1 T2 T3

FIGURE 4.1 ILLUSTRATES THE DIFFERENCE

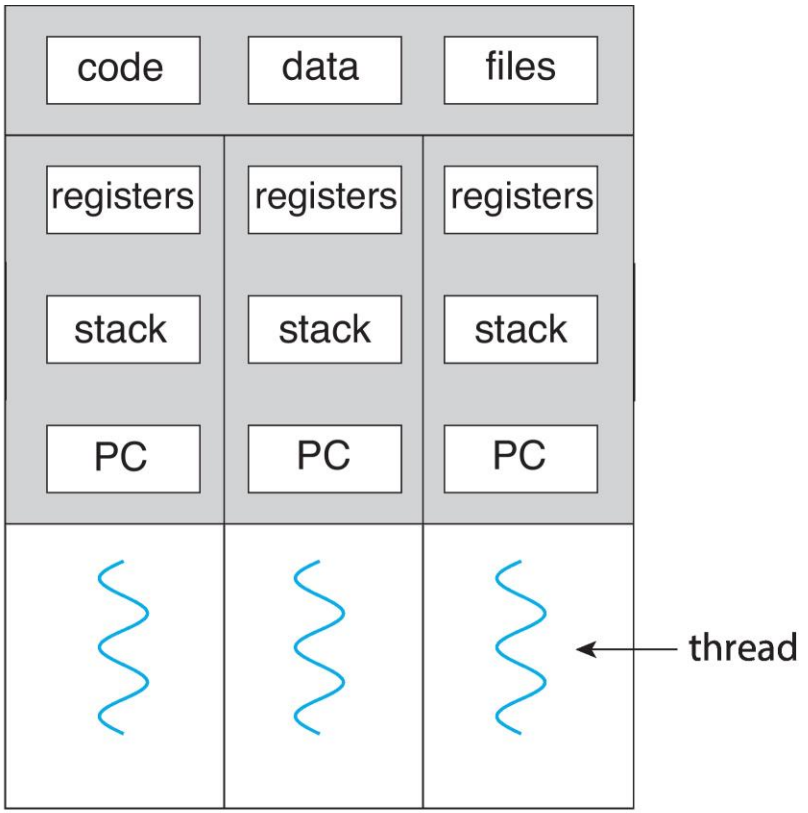
Summary: Efficient for multi-task local/network communication, requires filesystem/named space.



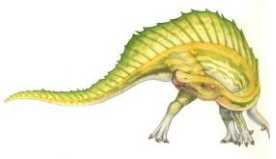
Single and Multithreaded Processes



single-threaded process



multithreaded process



OS KERNEL & KERNEL THREADS (LINUX EXAMPLE)



OS KERNEL: TYPICALLY MULTITHREADED

Operating system kernels are generally **multithreaded**. Tasks can be implemented by separate threads.

Essential for managing resources, scheduling, and concurrent system operations.

Contrassed call with <IMAGE>

**LINUX SYSTEM BOOT & KERNEL THREADS

During system boot, several kernel threads are created. Each performs a specific task.

ADAPTER

STARETED

Managing Devices

Memory Management

Interrupt Handling

Threads handle complex operations.

**DISPLAYING KERNEL THREADS via ps -ef

The **ps -ef** command displays kernel threads on a running system.

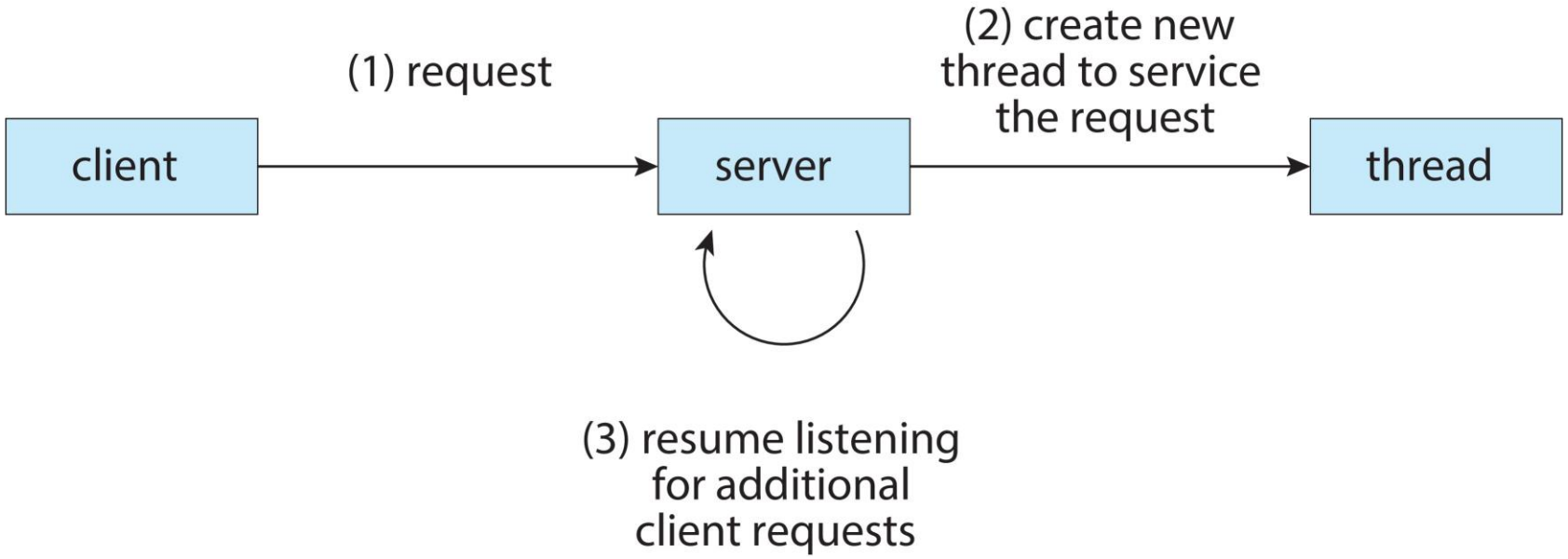
ID **kthreadd** (PID=2)
↳ serves as the parent of all other kernel threads.

PID	COMMAND	TASK EXAMPLE
2	kthreadd	Thread Creator
3	ksoftirqd	Interrupts
9	kworker	Async Work

Summary: Sockets efficiently manage local/network IPC, adapting the standard producer-consumer pattern handling.



Multithreaded Server Architecture





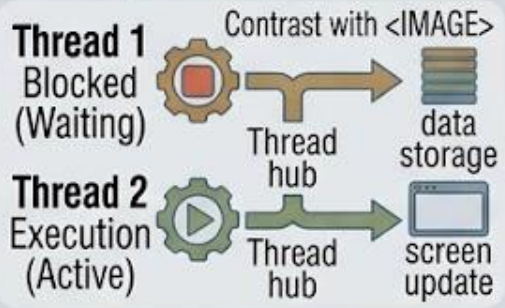
BENEFITS OF THREADS IN OPERATING SYSTEMS

**RESPONSIVENESS



Allows continued execution if part of process is blocked.

Critical for User Interfaces.
E.g., background data fetch while UI remains interactive.

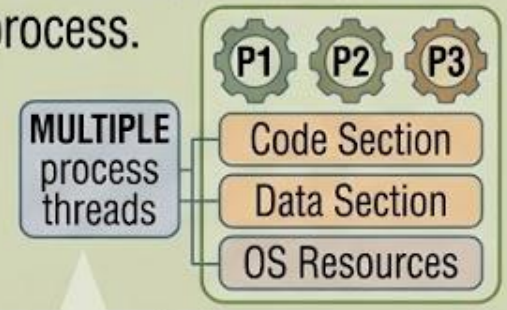


**RESOURCE SHARING



Threads share resources of the parent process.

More efficient than IPC methods (Shared Memory, Message Passing).

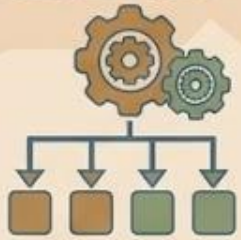


Contrassed with previous multi-process models.

**ECONOMY

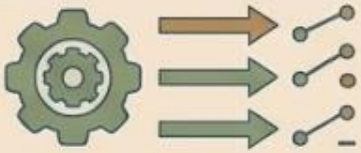


Thread creation is **chear** creation.
Thread switching has lower overhead than process context switching.



Heavy multi-state → (heavy cost)

Process Context Switch



Thread Context Switch

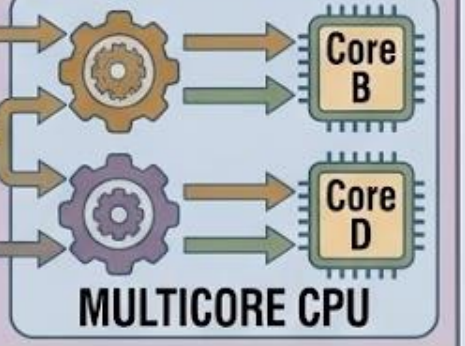
**SCALABILITY



Process can take advantage of multicore architectures.



Distribute threads for parallel processing.



Summary: Sockets efficiently manage local/network IPC, adapting the standard producer-consumer pattern handling.



Benefits

- **Responsiveness –** may allow continued execution if part of process is blocked, especially important for user interfaces
- **Resource Sharing –** threads share resources of process, easier than shared memory or message passing
- **Economy –** cheaper than process creation, thread switching lower overhead than context switching
- **Scalability –** process can take advantage of multicore architectures



MULTICORE PROGRAMMING: EVOLUTION & USE

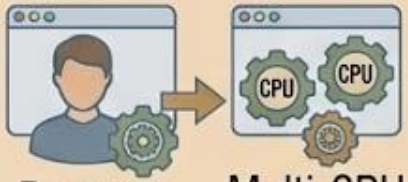
*1. MOTIVATION: THE NEED FOR MORE PERFORMANCE



Single-CPU systems reached efficiency limits.
Single-core performance plateaued.
The need for more computing performance.

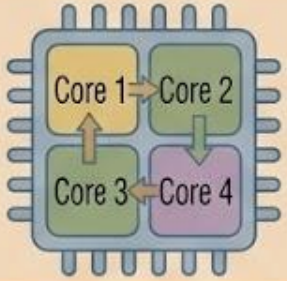


*2. EVOLUTION: MULTIPLE COMPUTING CORES ON ONE CHIP



Single is evolved into multi-CPU systems.

Multiple computing cores on a single processing chip. Such systems are called **MULTICORE SYSTEMS**.



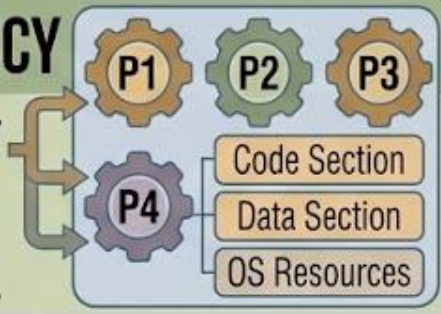
Each core appears as a **separate CPU** to the operating system.

*3. MECHANISM: EFFICIENT CORE UTILIZATION & CONCURRENCY



Multithreaded programming/Multicore programming provides a mechanism for **more efficient use** of these multiple computing cores and improved **concurrency**.

Distribute threads for parallel processing, modular design, direct resource access.



Summary: Efficient connection-oriented communication, adaptable to local or networked scope (references to <IMAGE>..



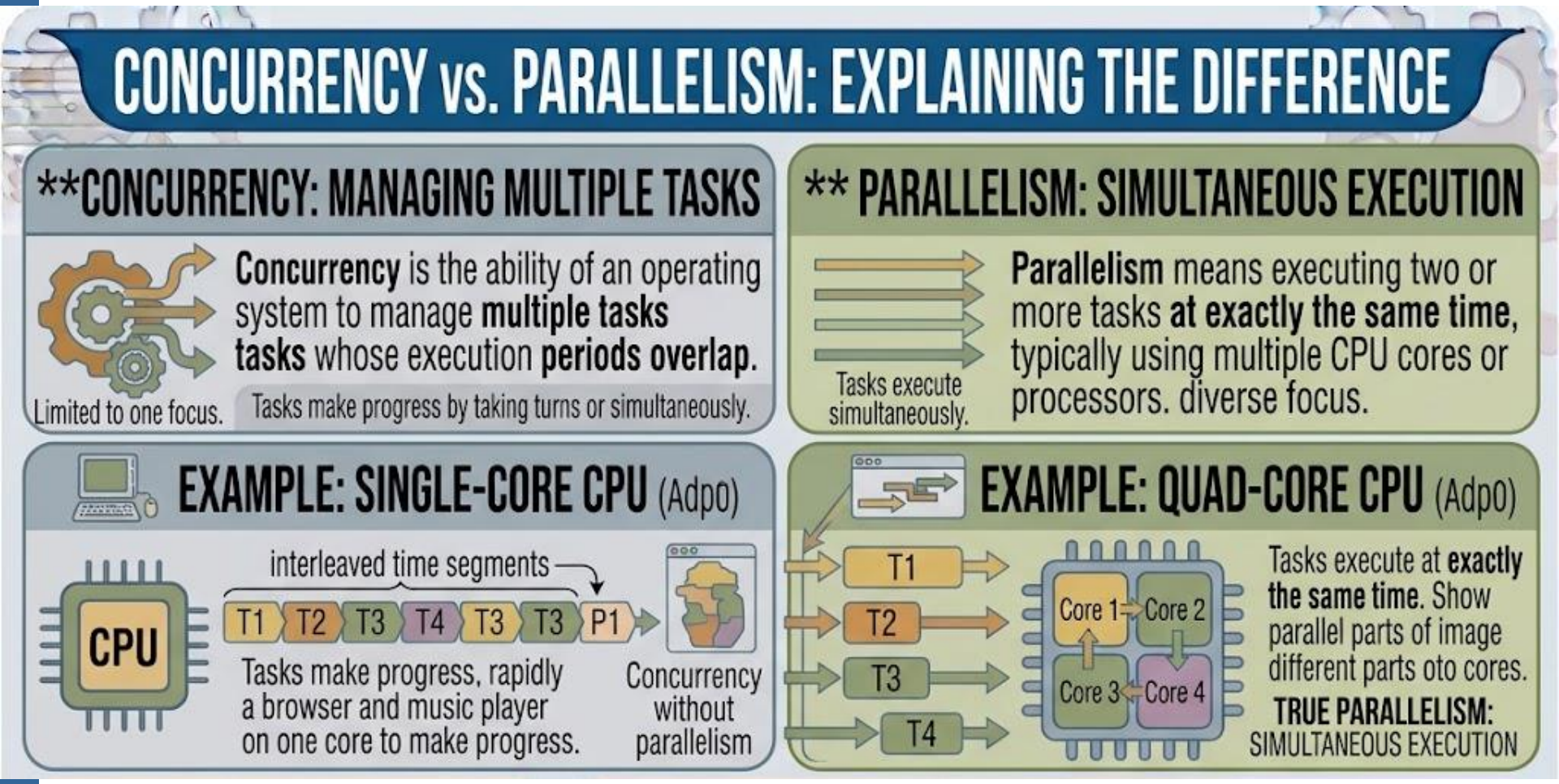
Multicore Programming

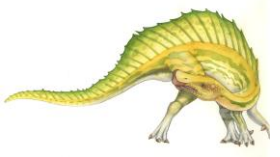
- The need for more computing performance, single-CPU systems evolved into multi-CPU systems.
- Multiple computing cores on a single processing chip where each core appears as a separate CPU to the operating system. Such systems are called **multicore systems**
- Multithreaded programming/Multicore programming provides a mechanism for more efficient use of these multiple computing cores and improved concurrency.





Concurrency vs. Parallelism

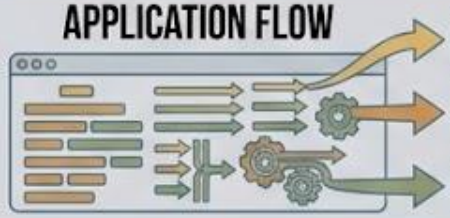




MULTICORE PROGRAMMING CHALLENGES: IDENTIFYING, BALANCING & SPLITTING

*1. IDENTIFYING CONCURRENT TASKS

This involves examining applications to find areas that can be divided into separate, concurrent tasks.



Examine code for concurrent potential.

*2. ACHIEVING BALANCED WORKLOAD

Tasks perform equal work of equal value. In some instances, a certain task may not contribute as much value to....

Even distribution of value and work. → **BALANCED**

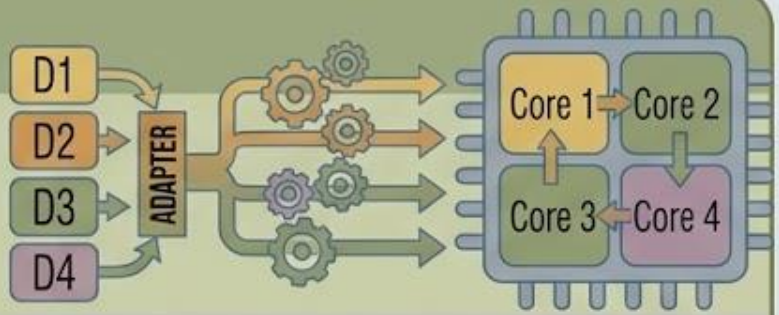


UNBALANCED

*3. DATA SPLITTING ACROSS CORES

The data accessed and manipulated by the tasks must be divided to run on separate cores.

TRUE PARALLELISM: SIMULTANEOUS EXECUTION



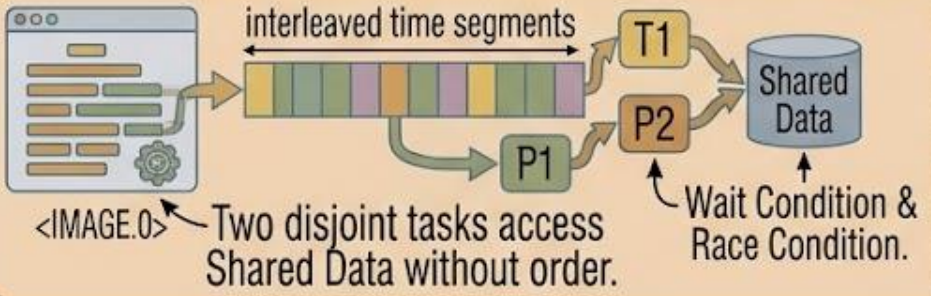
Summary: Concurrency without parallelism is possible (e.g., single-core interleaving). A parallel system must be concurrent.

MULTICORE PROGRAMMING CHALLENGES: DATA DEPENDENCY CONTINUED & TESTING

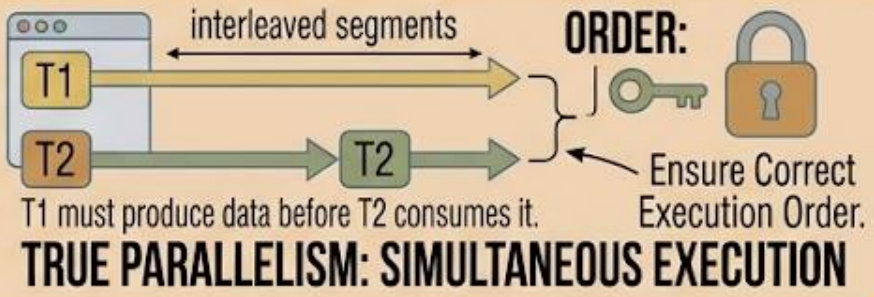
*4. DATA DEPENDENCY & SYNCHRONIZATION

When one task depends on data from another, programmers must ensure that the execution of the tasks is synchronized to accommodate the data dependency.

UNSYNCHRONIZED (Potential Race Condition)



SYNCHRONIZED (Data Flow Order)

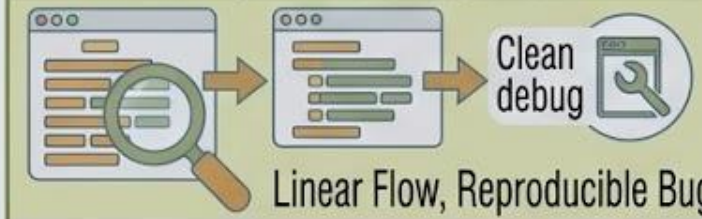


TRUE PARALLELISM: SIMULTANEOUS EXECUTION

**5. TESTING & DEBUGGING DIFFICULTY

Testing and debugging such concurrent programs is inherently more difficult.

SIMPLE PROGRAM



Linear Flow, Reproducible Bugs

CONCURRENT PROGRAM



Summary: Concurrency without parallelism is possible (e.g., single-core interleaving). A parallel system must be concurrent.



Multicore Programming challenges

- **Identifying tasks** :This involves examining applications to find areas that can be divided into separate, concurrent tasks.
- **Balance:** Tasks perform equal work of equal value. In some instances, a certain task may not contribute as much value to the overall process as other tasks.
- **Data splitting:** The data accessed and manipulated by the tasks must be divided to run on separate cores.
- **Data dependency:** When one task depends on data from another, programmers must ensure that the execution of the tasks is synchronized to accommodate the data dependency
- **Testing and debugging:** Testing and debugging such concurrent programs is inherently more difficult





Amdahl's Law

AMDAHL'S LAW: MULTICORE PERFORMANCE LIMITS

UNDERSTANDING THE CONCEPT (EXAMPLE)

VISUAL ADAPTER

100 SECONDS TOTAL EXECUTION TIME

20s — 20s SEQUENTIAL (non-parallelizable)

80s — 80s PARALLELIZABLE

Generic data handling (image_13.png, image_16.png)

**1 CPU CORE: rapidly rapidly rapidly switching.

CPU

Linear: 20s S

interleaved time segments: Linear 80s P

Sequential Execution

Full 100s execution

ExecutionTime(1 Core)

**MULTIPLE CORES (e.g., Infinite)

Core 1, Core 2, Core 3, Core 4

Processors

20s S

CONCURRENT PROGRAM

T1 T2 T3...

T2 T3 ... T4

T5

Non-Deterministic, Race Conditions, Deadlocks

0s

Min Time: 20s

Execute complex

TRUE PARALLELISM: SIMULTANEOUS EXECUTION

** MAXIMUM SPEEDUP CALCULATION



Generic parts (image_13.png) (image_16.png) to
S (Serial) r: **P** (parallel
N (Processing cores)

$$\text{Speedup} = \frac{\text{Total Time}}{\text{Min Sequential Time}} \qquad \text{Speedup} = \frac{100s}{20s} = 5x$$

Maximum speedup limited to 5x

Even with infinite cores, sequential part dominates.



Amdahl's Law

AMDAHL'S LAW: FORMULA & NUMERICAL EXAMPLE

**AMDAHL'S LAW FORMULA

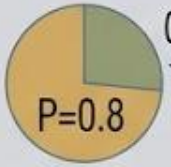


Limited ceiling



$$\text{Speedup } S(N) = \frac{1}{[(1 - P) + (P / N)]}$$

S(N) = overall speedup using N processors
P = fraction of the program that can be parallelized
1-P = fraction that must execute sequentially
N = number of processors or CPU cores



P=0.8, 1-P=0.2

Generic parts (image_13.png, image_16.png) to
S (Serial portion)
P (Parallel portion)
N (Processing cores)

**NUMERICAL EXAMPLE (A program taking 100s, 80% parallelized)



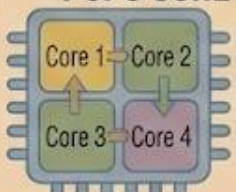
Limited to one focus, but multi-pronged flow.

Calculate speedup and new execution time when using 4 processors.

20s S | 80s P

ExecutionTime(1 Core)

**1 CPU CORE (100s)

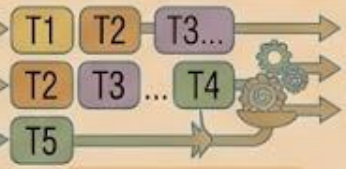


Processors

20s S

Min Time: 20s

CONCURRENT PROGRAM



Execute complex

Non-Deterministic, Race Conditions, Deadlocks

0s

TRUE PARALLELISM: SIMULTANEOUS EXECUTION

*3. CALCULATION & RESULTS



Step 1: Identify Values): P=0.8, N=4
Step 2: Apply Amdahl's Law:
Speedup S(4) = 1 / [(1 – 0.8) + (0.8 / 4)]
= 1 / 0.4

Step 3: Calculate New Execution Time):
$T_{new} = T_{original} / S(4)$
= 100 / 2.5
T_{new} = 40 seconds

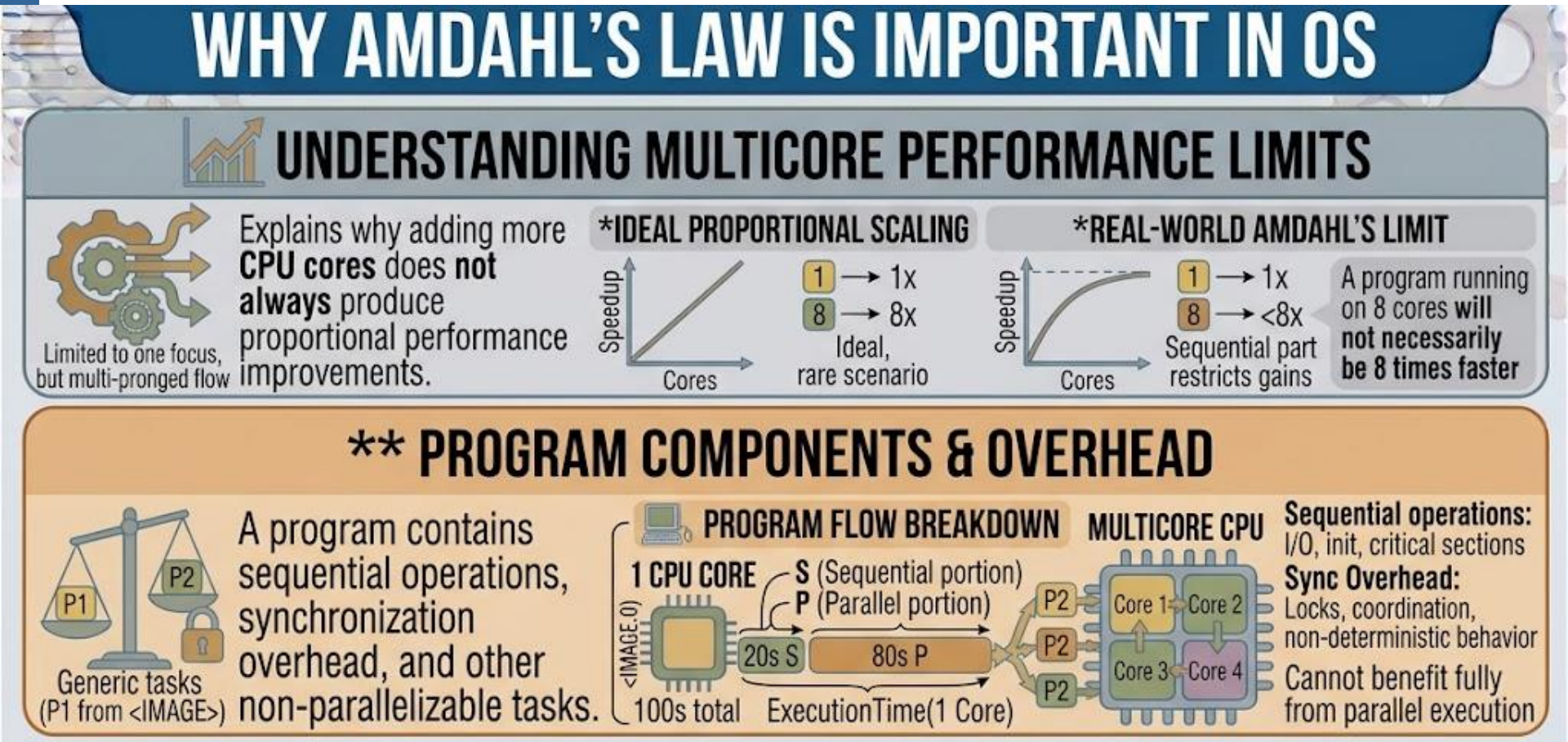
Non-reproducible bugs
Many interleaving states
Generic multi-threaded flow
Linear flow for 1 core

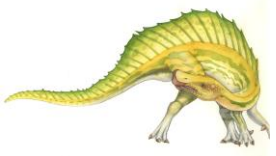


2.5x speedup
T_{new} = 40 seconds



Amdahl's Law


WHY AMDAHL'S LAW IS IMPORTANT IN OS
UNDERSTANDING MULTICORE PERFORMANCE LIMITS
Limited to one focus, but multi-pronged flow
Explains why adding more CPU cores does not always produce proportional performance improvements.
*IDEAL PROPORTIONAL SCALING
Speedup
Cores
1 → 1x
8 → 8x
Ideal, rare scenario
*REAL-WORLD AMDAHL'S LIMIT
Speedup
Cores
1 → 1x
8 → <8x
Sequential part restricts gains
A program running on 8 cores will not necessarily be 8 times faster
** PROGRAM COMPONENTS & OVERHEAD
P1
P2
Generic tasks (P1 from <IMAGE>)
A program contains sequential operations, synchronization overhead, and other non-parallelizable tasks.
PROGRAM FLOW BREAKDOWN
1 CPU CORE
<IMAGE.0)
S (Sequential portion)
P (Parallel portion)
20s S
80s P
100s total
ExecutionTime(1 Core)
MULTICORE CPU
P2
P2
P2
Core 1
Core 2
Core 3
Core 4
Sequential operations: I/O, init, critical sections
Sync Overhead: Locks, coordination, non-deterministic behavior
Cannot benefit fully from parallel execution





Amdahl's Law

- Identifies performance gains from adding additional cores to an application that has both serial and parallel components
- *S* is serial portion
- *N* processing cores

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- That is, if application is 75% parallel / 25% serial, moving from 1 to 2 cores results in speedup of 1.6 times
- As *N* approaches infinity, speedup approaches 1 / *S*

Serial portion of an application has disproportionate effect on performance gained by adding additional cores



MULTICORE PROGRAMMING: TYPES OF PARALLELISM

*1. DATA PARALLELISM

Distributes subsets of the same data across multiple cores, same operation on each.

CONCURRENCY with Data Flow order
Generic data handling <_13.png, image_16.png

EXAMPLE: SUM OF ARRAY SIZE N ON TWO CORES

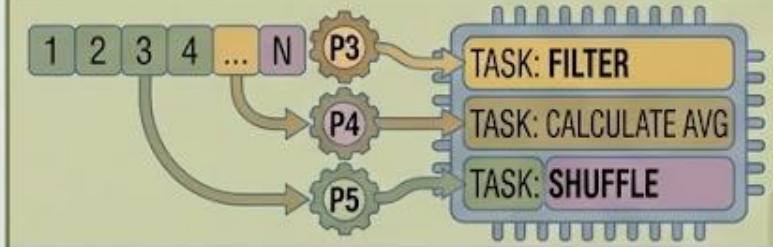


**2. TASK PARALLELISM: Distributing threads across cores, each thread performing unique operation.

simultaneours work on each **Concurrently** but not in parallel (<IMAGE_2>, <IMAGE-3>).

EXAMPLE: DIFFERENT TASKS ON SAME ARRAY IN PARALLEL

Concurrent Program



Distribute threads for parallel processing, **modular design**, direct resource access.

interleaved segments
Non-Deterministic,
Race Conditions,
Deadlocks



Execute complex

TRUE PARALLELISM: SIMULTANEOUS EXECUTION





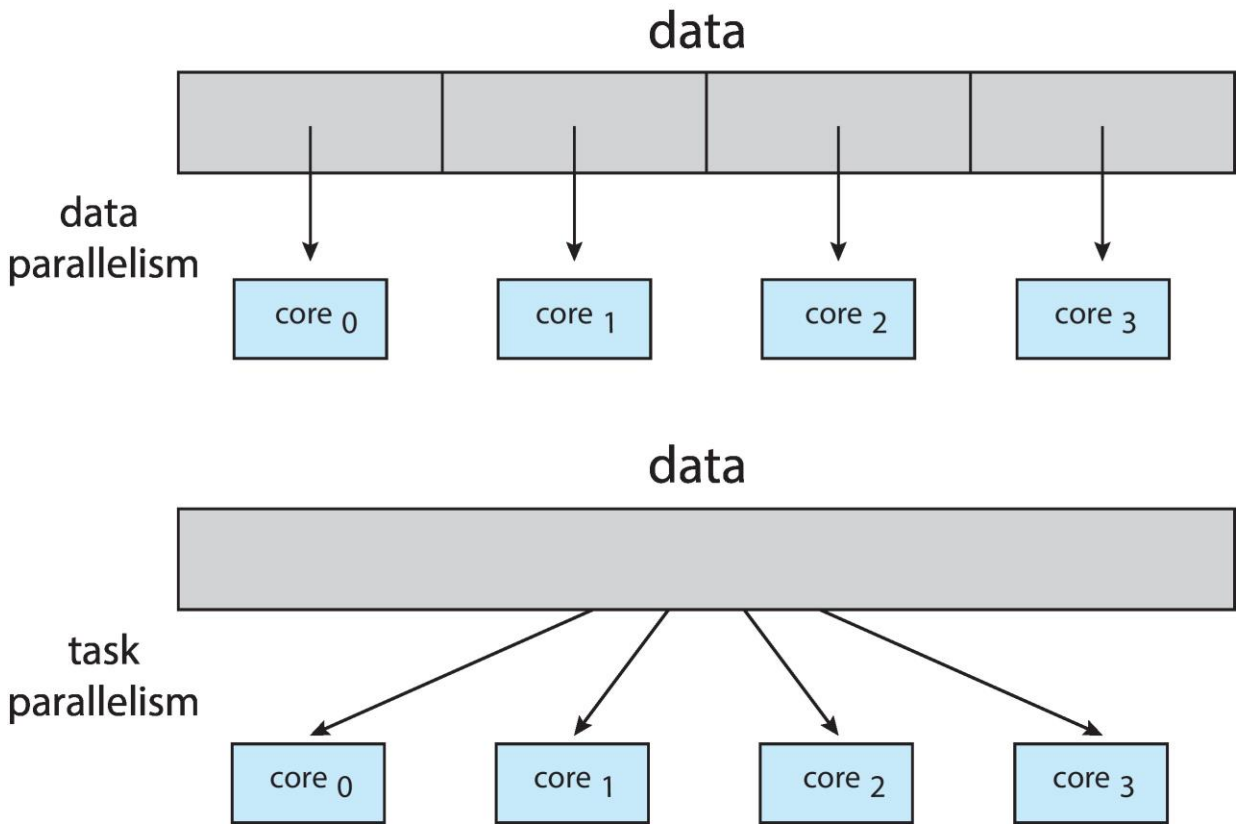
Multicore Programming

- Types of parallelism
  - **Data parallelism** – distributes subsets of the same data across multiple cores, same operation on each.
  - Example: Sum of an Array of size N on two different cores
  - **Task parallelism** – distributing threads across cores, each thread performing unique operation
  - Example: different tasks on the same array in parallel





Data and Task Parallelism







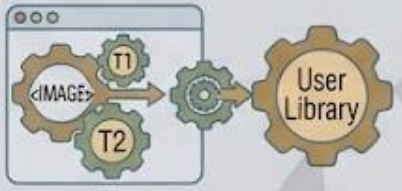
User Threads and Kernel Threads

USER THREADS
(Library-managed)

**Thread Management

Managed by user-level threads library.

Library handles scheduling, synchronization



thread hub

Generic data handling (image_16.png)

**Three Primary Thread Libraries:



Icons from <IMAGE> as generalized.

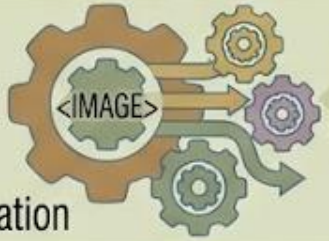
- POSIX Pthreads
- Windows threads
- **Java** threads

KERNEL THREADS
(System-managed)

**Kernad Management

Supported and managed directly by the OS Kernel.

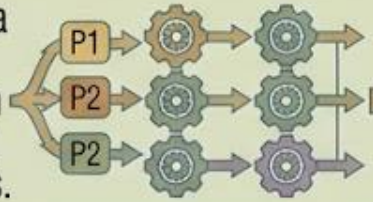
Kernel handles scheduling, synchronization



Generic parts (image_13.png, image_16.png)

**Supported by Virtually All General-Purpose OSs:

Generalized data flow derived & concurrent from generic parts of multi-CPU flows.



- Windows
- Linux
- Mac OS X
- iOS
- Android

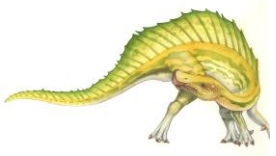
User thread

Word processor

1. Spelling

2. Changing font

while writing programme for this application there are libraries that will help the programmer to manage these threads



KERNEL THREAD CREATION & MANAGEMENT: DYNAMIC SYSTEM BEHAVIOR

SYSTEMS ARCHITECTURE OVERVIEW

Understanding How Kernel Threads Work

Kernel threads are created and managed by the operating system kernel, in response to requests from applications or system components. The OS does not maintain one fixed pool.

1. Are Kernel Threads Already Present?

Startup

Computer Boot: OS creates threads for its own tasks (e.g., system tasks).

Application Start

As applications and services start, additional threads are created as needed.

The OS dynamically creates threads to perform system tasks and service application requests.

Dynamic Creation Example

Kernel threads are generated dynamically based on application need.

Dynamic Process:

FIFO Order.

U1

A browser may request creation of threads (e.g., for downloads).

U2

...or user input handling.

Application uses API to request threads, which maps to kernel creation.

Mapping to Kernel-Supported Threads

(1) USER THREAD REQUESTS

Application uses a threading API to request task execution.

(2) THREAD LIBRARY (API)

Library identifies parallel sections and makes requests.

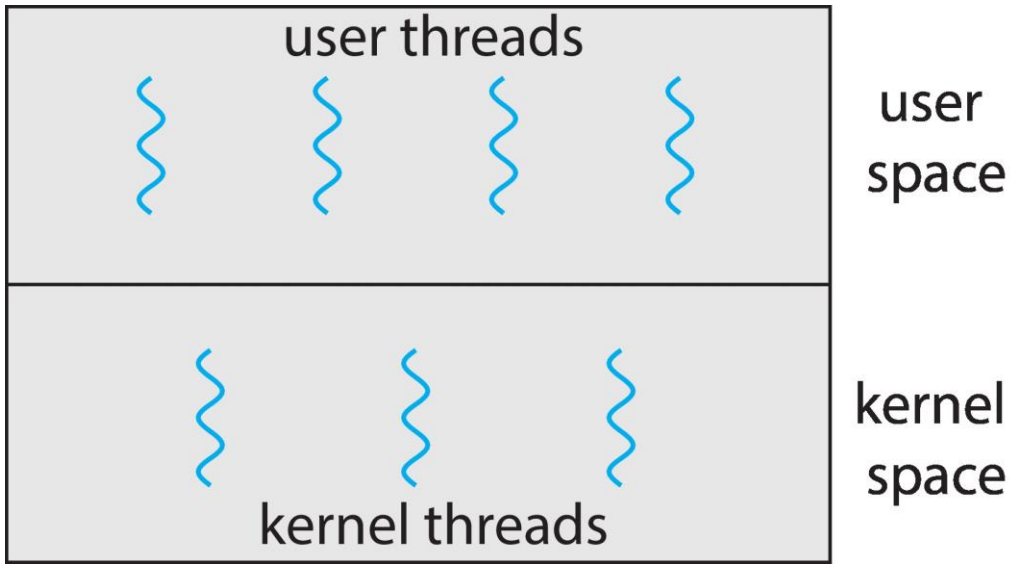
(3) OS KERNEL ACTION

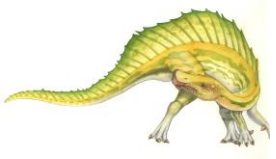
OS kernel creates or sets up corresponding kernel-supported (e.g., one-to-one, many-to-many).

DYNAMIC KERNEL THREAD CREATION: VISUAL DEEP DIVE



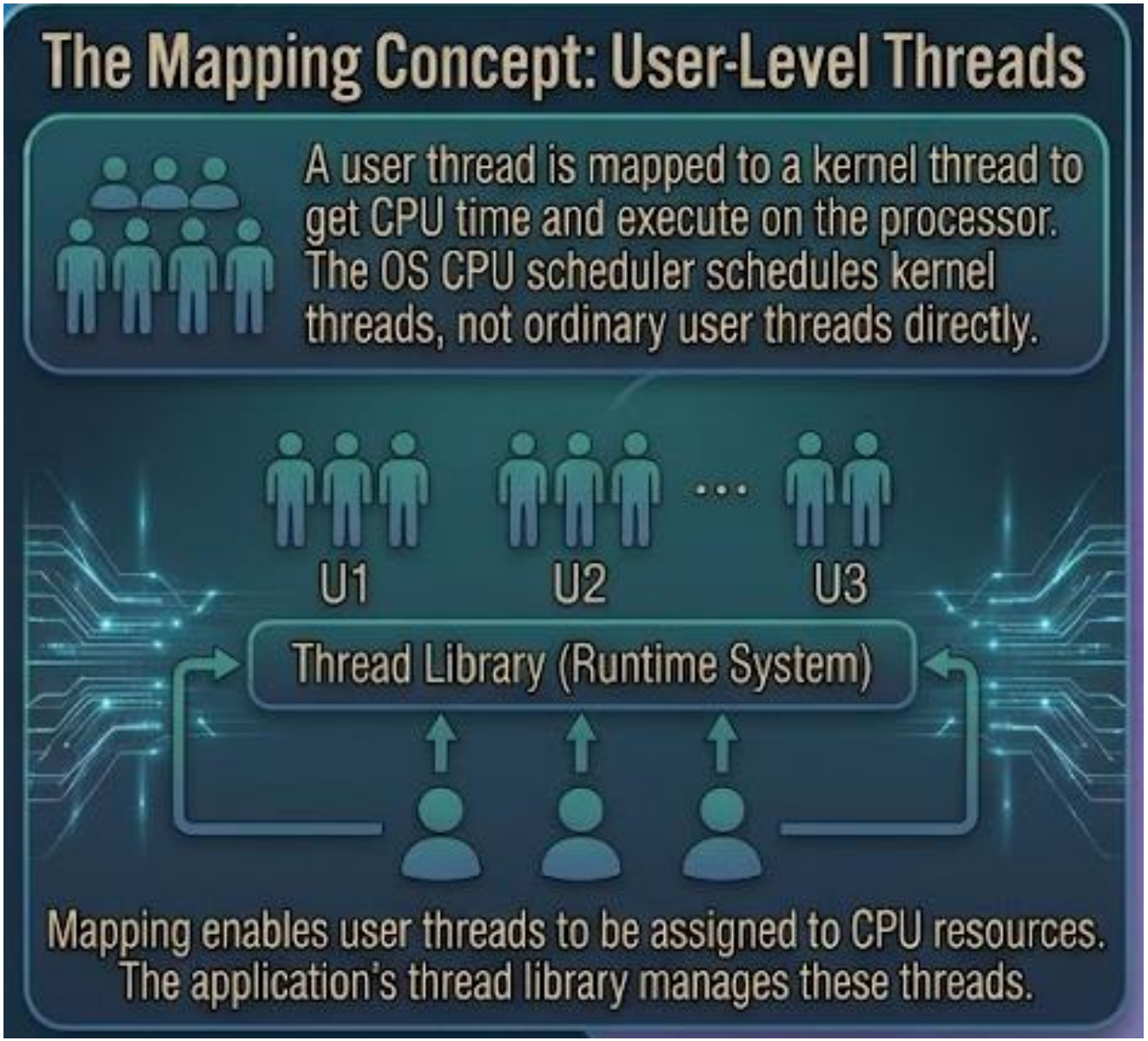
User and Kernel Threads

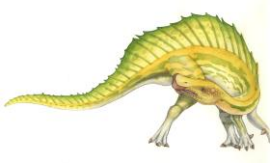






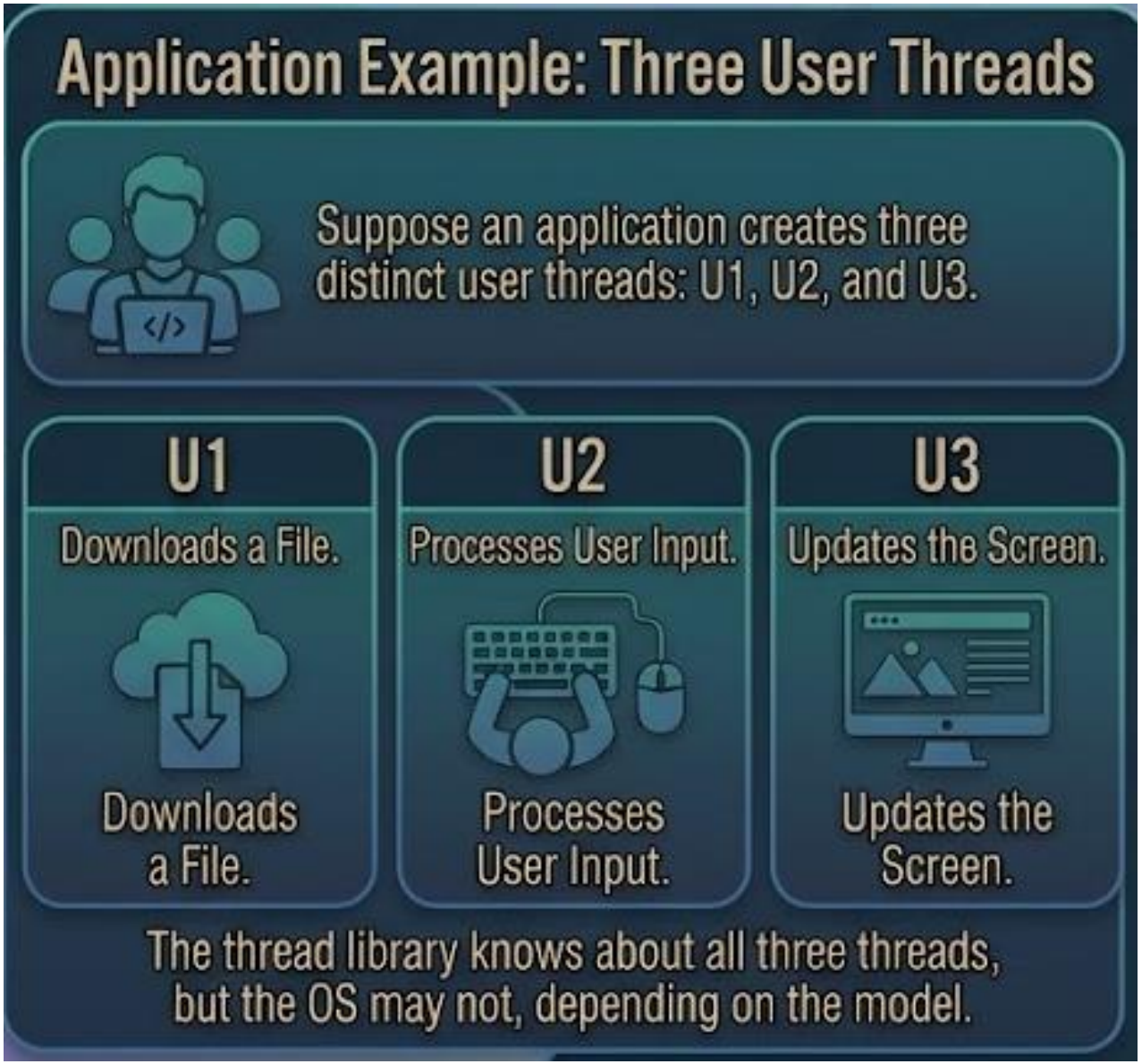
Why User and Kernel Threads are mapped

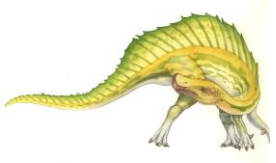






Why User and Kernel Threads are mapped







Multithreading Models

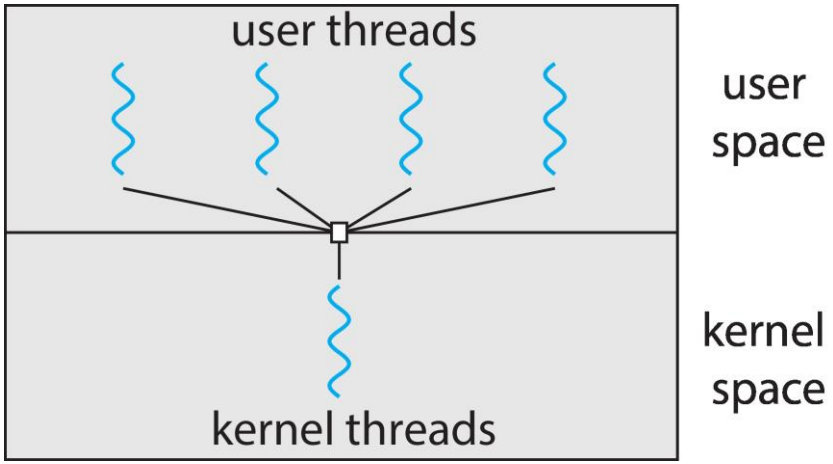
- Many-to-One
- One-to-One
- Many-to-Many

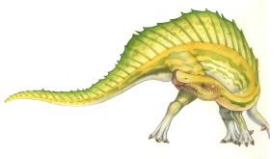




Many-to-One

- Many user-level threads mapped to single kernel thread
- One thread blocking causes all to block
- Multiple threads may not run in parallel on multicore system because only one may be in kernel at a time
- Few systems currently use this model
- Examples:
  - **LINUX**

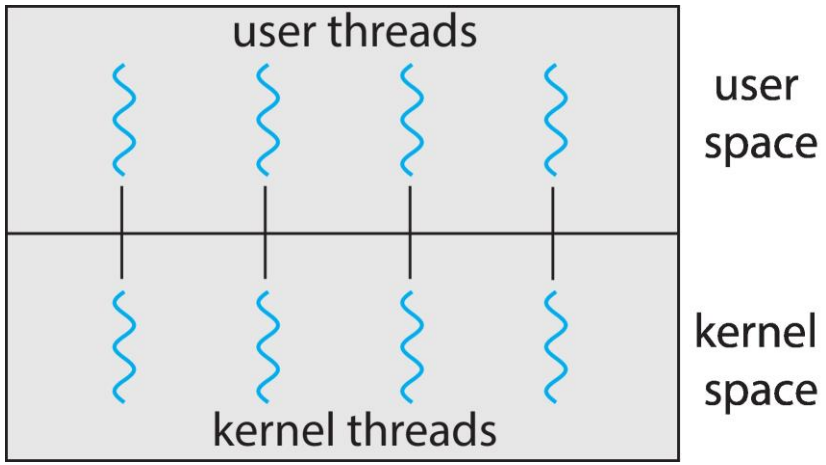






One-to-One

- Each user-level thread maps to kernel thread
- Creating a user-level thread creates a kernel thread
- More concurrency than many-to-one
- Number of threads per process sometimes restricted due to overhead
- Examples
  - Windows
  - Linux

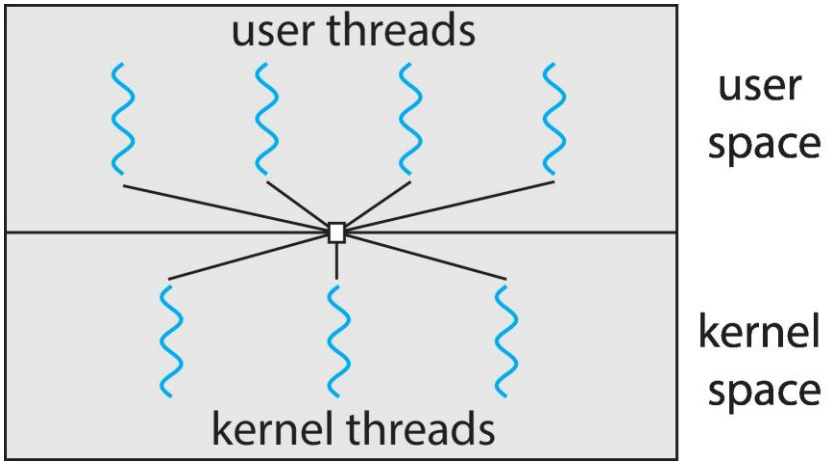


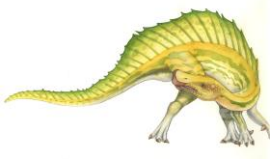




Many-to-Many Model

- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads
- Windows with the *ThreadFiber* package
- Otherwise not very common

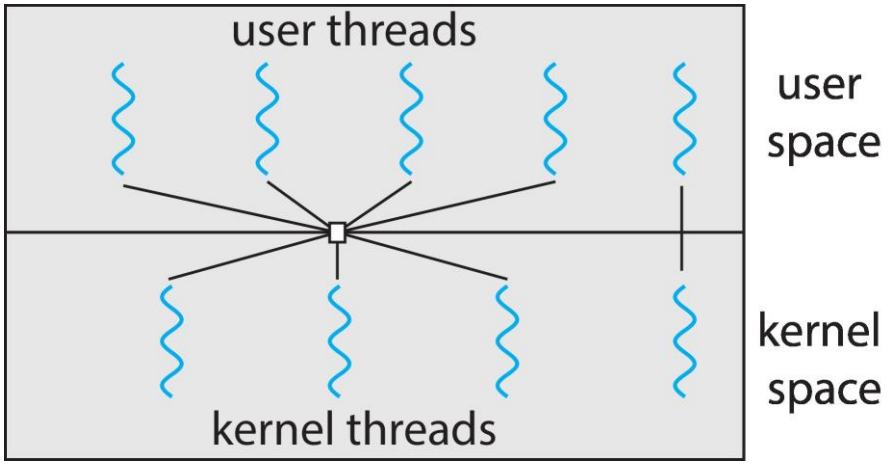


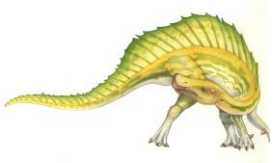




Two-level Model

- Similar to M:M, except that it allows a user thread to be **bound** to kernel thread





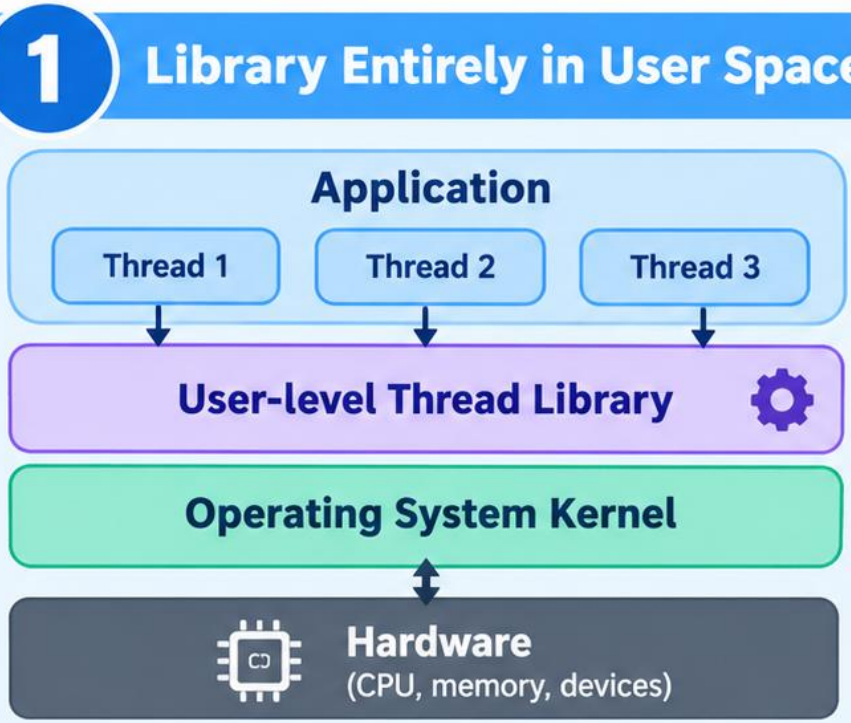
Thread Library

Provides programmer with API for creating and managing threads

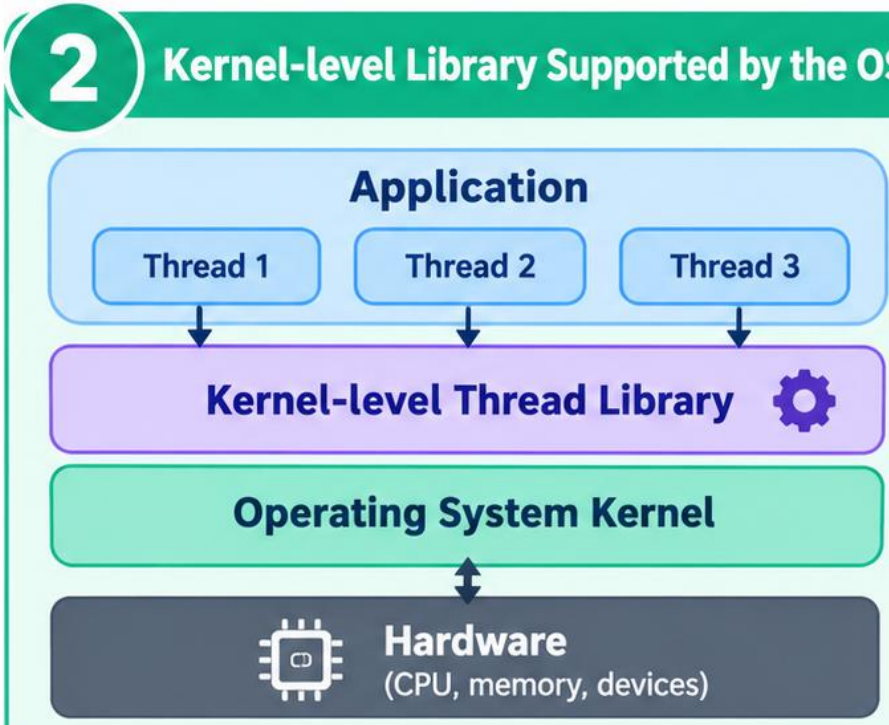
API

Two primary ways of implementing

1 Library Entirely in User Space



2 Kernel-level Library Supported by the OS





Thread Libraries

Feature	User Threads	Kernel Threads
Managed by	User-level thread library	OS kernel
Kernel awareness	Kernel may not know individual threads exist	Kernel knows each thread
Management overhead	Usually lower	Usually higher
Scheduling	Thread library, depending on model	Kernel scheduler
Parallel execution	Not in a many-to-one model	Supported on multiple cores
Blocking system call	May block the entire process in a many-to-one model	Generally, blocks only the calling thread
Example	Threads managed by a user-level runtime	POSIX threads implemented using kernel-supported threads on Linux



Pthreads: POSIX Threads

User-Level or Kernel-Level

May be provided as either type, though modern implementations are often hybrid.



Managed entirely by a library in user space.
Fast context switching.
Transparent to kernel.

Scheduled and managed by the OS kernel.
Blocks can be independent.
True parallelism on multi-cores.

POSIX Standard (IEEE 1003.1c)

A defined API for thread creation and synchronization.
Part of the broader POSIX standards.

Portable Operating System Interface

Specification, Not Implementation

Specification (What to do)

Defines the API signatures, return values, expected behaviors (e.g., locking, signal handling).

Implementation (How to do it)

Up to library developers and OS kernel developers (e.g., NPTL, LinuxThreads, various Unix variants).

API specifies behavior of the thread library, implementation is up to development of the library

Common in UNIX Operating Systems



Widely used in Linux, macOS, FreeBSD, and other Unix-like systems for multi-multi-threaded applications.



Pthreads

- May be provided either as user-level or kernel-level
- A POSIX standard (IEEE 1003.1c) API for thread creation and synchronization
- ***Specification***, not ***implementation***
- API specifies behavior of the thread library, implementation is up to development of the library
- Common in UNIX operating systems (Linux & Mac OS X)





Pthreads Example

```
#include <pthread.h>
#include <stdio.h>

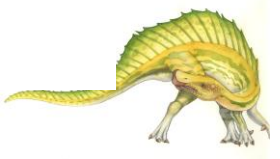
#include <stdlib.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
   pthread_t tid; /* the thread identifier */
   pthread_attr_t attr; /* set of thread attributes */

   /* set the default attributes of the thread */
   pthread_attr_init(&attr);
   /* create the thread */
   pthread_create(&tid, &attr, runner, argv[1]);
   /* wait for the thread to exit */
   pthread_join(tid,NULL);

   printf("sum = %d\n",sum);
}
```





Pthreads Example (Cont.)

```
/* The thread will execute in this function */
void *runner(void *param)
{
   int i, upper = atoi(param);
   sum = 0;

   for (i = 1; i <= upper; i++)
      sum += i;

   pthread_exit(0);
}
```





Pthreads Code for Joining 10 Threads

```
#define NUM_THREADS 10

/* an array of threads to be joined upon */
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
   pthread_join(workers[i], NULL);
```



WINDOWS THREADS

Key Windows Thread Functions

Functions for thread lifecycle management

- CreateThread
- ExitThread
- ResumeThread

Synchronization Techniques

API

- Mutexes
- Semaphores
- Waitable Timers
- Critical Sections

Preventing race conditions and ensuring proper execution

API

Pthreads API

Similarities to

Application	Application	Application	Application
pthread_create()	CreateThread()	pthread_create()	CreateThread()
pthreads library	Windows API	pthreads library	Windows API
Kernel	Kernel	Kernel	Kernel
		Pthreads	Windows Threads

The general process of thread creation, management, and synchronization follows a similar logical flow.



Windows Multithreaded C Program

```
#include <windows.h>
#include <stdio.h>
DWORD Sum; /* data is shared by the thread(s) */

/* The thread will execute in this function */
DWORD WINAPI Summation(LPVOID Param)
{
   DWORD Upper = *(DWORD*)Param;
   for (DWORD i = 1; i <= Upper; i++)
      Sum += i;
   return 0;
}
```





Windows Multithreaded C Program (Cont.)

```
int main(int argc, char *argv[])
{
   DWORD ThreadId;
   HANDLE ThreadHandle;
   int Param;

   Param = atoi(argv[1]);
   /* create the thread */
   ThreadHandle = CreateThread(
      NULL, /* default security attributes */
      0, /* default stack size */
      Summation, /* thread function */
      &Param, /* parameter to thread function */
      0, /* default creation flags */
      &ThreadId); /* returns the thread identifier */

    /* now wait for the thread to finish */
   WaitForSingleObject(ThreadHandle,INFINITE);

   /* close the thread handle */
   CloseHandle(ThreadHandle);

   printf("sum = %d\n",Sum);
}
```



Java Threads: JVM Management & Creation

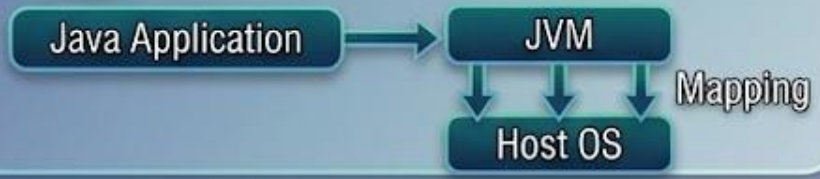
Managed by the JVM

Threads are fully managed and scheduled by the Java Virtual Machine.



OS Thread Mapping

Java threads typically map to the underlying OS's native thread model for efficiency.



Standard Practice

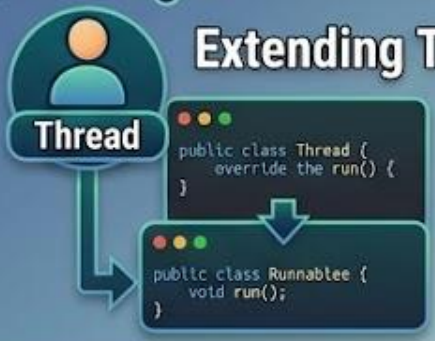
Standard practice is to implement the **Runnable** interface.



Java Thread Creation Methods

PREFERRED

Extending Thread Class



Create a new class that extends `Thread` and override the `run()` method.

Implementing Runnable Interface



Create a class implementing the `Runnable` interface and pass it to a new `Thread` instance.



Java Threads

Implementing Runnable interface:

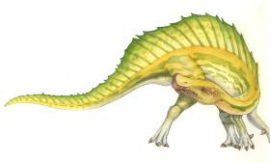
```
class Task implements Runnable
{
   public void run() {
      System.out.println("I am a thread.");
   }
}
```

Creating a thread:

```
Thread worker = new Thread(new Task());
worker.start();
```

Waiting on a thread:

```
try {
   worker.join();
}
catch (InterruptedException ie) { }
```





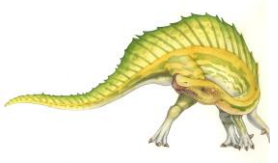
Java Executor Framework

- Rather than explicitly creating threads, Java also allows thread creation around the Executor interface:

```
public interface Executor
{
   void execute(Runnable command);
}
```

- The Executor is used as follows:

```
Executor service = new Executor;
service.execute(new Task());
```





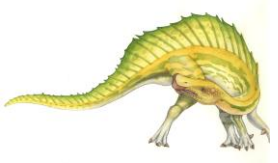
Java Executor Framework

```
import java.util.concurrent.*;

class Summation implements Callable<Integer>
{
   private int upper;
   public Summation(int upper) {
      this.upper = upper;
   }

   /* The thread will execute in this method */
   public Integer call() {
      int sum = 0;
      for (int i = 1; i <= upper; i++)
         sum += i;

      return new Integer(sum);
   }
}
```





Java Executor Framework (Cont.)

```
public class Driver
{
 public static void main(String[] args) {
    int upper = Integer.parseInt(args[0]);

    ExecutorService pool = Executors.newSingleThreadExecutor();
    Future<Integer> result = pool.submit(new Summation(upper));

    try {
        System.out.println("sum = " + result.get());
    } catch (InterruptedException | ExecutionException ie) { }
  }
}
```



EXPLICIT VS IMPLICIT THREADING



EXPLICIT THREADING

A programming approach where the programmer directly creates, manages, and synchronizes threads using threading APIs or libraries.

MANUAL CONTROL



IMPLICIT THREADING

An approach where the programmer specifies concurrent tasks, while a runtime system, compiler, or thread library manages creation, scheduling, and execution.

AUTOMATED SYSTEM

High Control, High Overhead

Who Manages Threads?



The programmer

How?



Step 1: Explicitly Creating Threads



Step 2: Assigning Tasks



Step 3: Handling Synchronization

Example

```
{
    {
        (    );
}
    );
}
```

`pthread_create()`

Thread in Java

Who Manages Threads?



The runtime system or compiler

How?



Step 1: The Programmer Defines Tasks



Step 2: The System Manages Execution



Mechanism: Uses mechanisms like thread pools or parallel frameworks.

Example

```
{
if (    ){
    #pragma omp parallel;
}
```

OpenMP directives in C/C++

`#pragma omp parallel`

THREAD MANAGEMENT OVERVIEW

IMPLICIT THREADING

Implicit Threading Overview

PROBLEM with Explicit Threads:
Increasing thread counts make program correctness more difficult for programmers.

IMPLICIT THREADING SOLUTION:
Thread creation and management handled by compilers and run-time libraries rather than programmers.

1. THREAD POOLS



A collection of threads created once. Tasks are assigned to available threads. Efficient resource use.

2. FORK-JOIN



A "master" thread forks work into multiple sub-threads. When finished, threads join back to the master. Common for divide-and-conquer problems.

3. OPENMP

```
#pragma omp
...
  for (int i=+) {
    parallel omp;
  }
}
```

A set of compiler directives & library routines. Programmers specify parallel regions, and the compiler handles thread creation. Widely used for data parallelism.

FIVE METHODS EXPLORED



4. GRAND CENTRAL DISPATCH

Implemented as a library. Developers submit tasks (blocks of code) to dispatch queues. GCD manages thread creation & scheduling on Apple hardware.

5. INTEL THREADING BUILDING BLOCKS



A C++ template library. Provides parallel versions of code of standard patterns (e.g., parallel loops, loops, scalable memory allocation, pipelines). Optimizes code for Intel processors.

THREAD POOLS

TECHNICAL DEEP DIVE

ADVANTAGES

SPEED

Slightly faster to service a request. Avoids the overhead of creating a new thread.

THREAD BOUNDARY

≈ Number lover limit

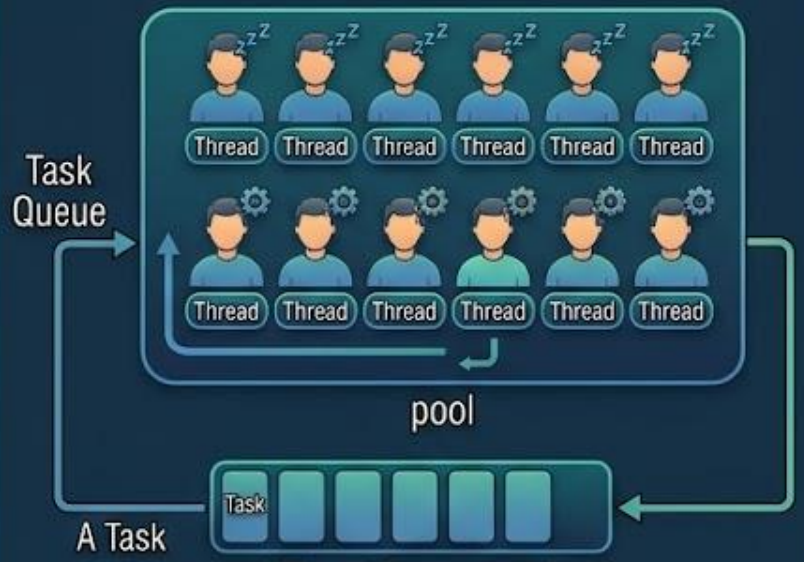
Keeps the application's thread count predictable. The pool size acts as an upper bound.

TASK SEPARATION

Separates task definition from thread creation mechanics. Allows different run strategies.

AWAITING WORK

What it does:



A Thread changes to tael and moves to work on it, then returns.

ADVANTAGES

EXAMPLE: PERIODIC SCHEDULING



Tasks can be scheduled to run on a predictable, periodic basis.

Windows API Supports Thread Pools

CreateThreadpool()
SubmitThreadpoolWork()
SubmitThreadpoolWork()
PThreadpoolWork()

The Windows API provides direct support with dedicated functions for efficient pool management.

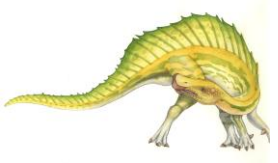
THREAD POOL OVERVIEW



Thread Pools

- Create a number of threads in a pool where they await work
- Advantages:
  - Usually slightly faster to service a request with an existing thread than create a new thread
  - Allows the number of threads in the application(s) to be bound to the size of the pool
  - Separating task to be performed from mechanics of creating task allows different strategies for running task
    - i.e.,Tasks could be scheduled to run periodically
- Windows API supports thread pools:

```
DWORD WINAPI PoolFunction(AVOID Param) {
    /*
    * this function runs as a separate thread.
    */
}
```





Java Thread Pools

- Three factory methods for creating thread pools in Executors class:
  - `static ExecutorService newSingleThreadExecutor()`
  - `static ExecutorService newFixedThreadPool(int size)`
  - `static ExecutorService newCachedThreadPool()`





Java Thread Pools (Cont.)

```
import java.util.concurrent.*;

public class ThreadPoolExample
{
public static void main(String[] args) {
   int numTasks = Integer.parseInt(args[0].trim());

   /* Create the thread pool */
   ExecutorService pool = Executors.newCachedThreadPool();

   /* Run each task using a thread in the pool */
   for (int i = 0; i < numTasks; i++)
      pool.execute(new Task());

   /* Shut down the pool once all threads have completed */
   pool.shutdown();
}
```



FORK-JOIN PARALLELISM

TECHNICAL DEEP DIVE

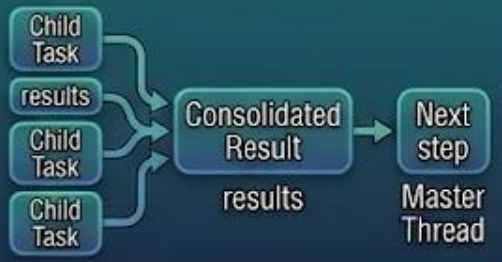
1. Forking Tasks



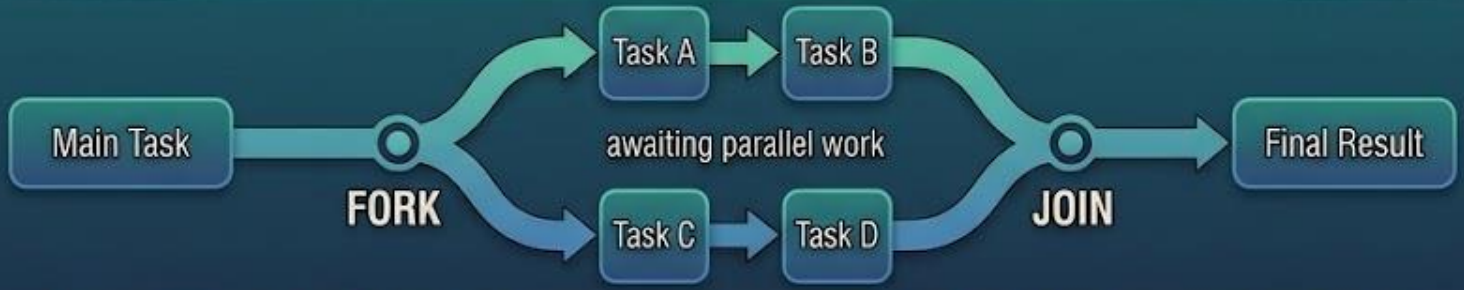
Multiple threads or tasks are forked from a main process.
The "fork" operation splits complex work.

2. Joining Results

The "join" operation waits for all forked threads to complete, merging their results.



The Fork-Join Model



Allows efficient use of multiple processing cores for divio divide-and-conquer strategies.

KEY BENEFITS

PARALLELISM: Multiple tasks execute simultaneously

DIVIDE-AND-CONQUER: Complex tasks are broken down

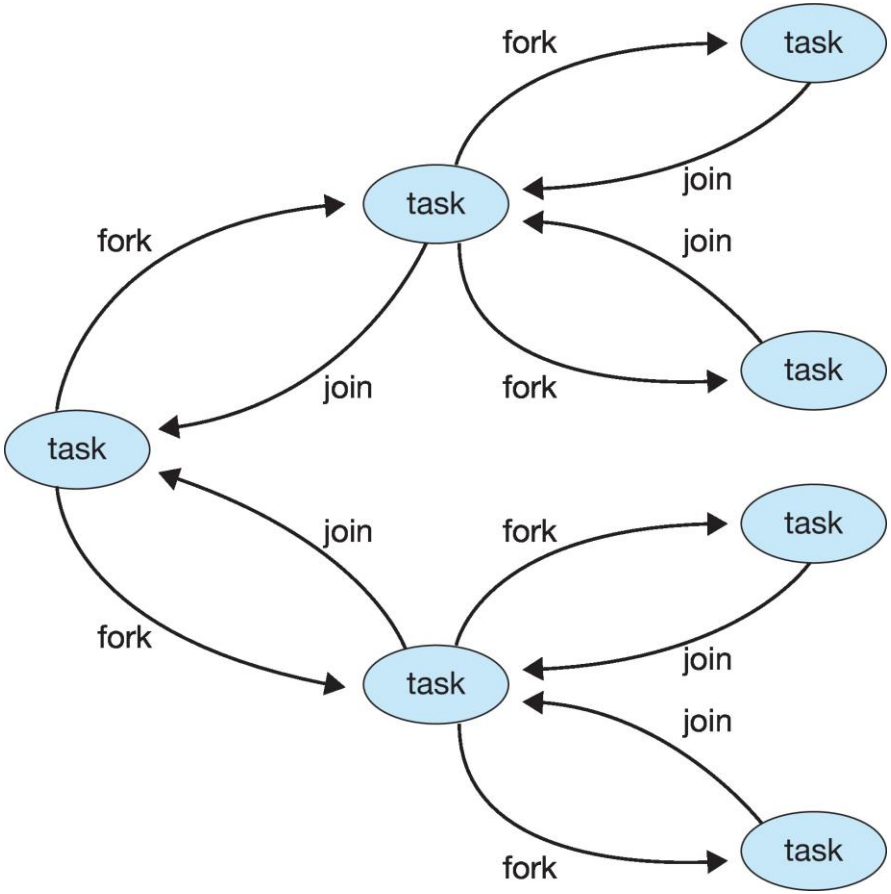
RESULTS CONSOLIDATION: A simple way to combine data

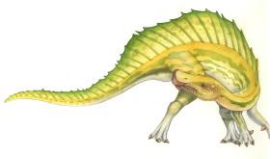
SCALABILITY: Efficiently uses cores

FORK-JOIN PARALLELISM OVERVIEW



Fork-Join Parallelism







Fork-Join Parallelism

- General algorithm for fork-join strategy:

```
Task(problem)
   if problem is small enough
     solve the problem directly
   else
     subtask1 = fork(new Task(subset of problem)
     subtask2 = fork(new Task(subset of problem)

     result1 = join(subtask1)
     result2 = join(subtask2)

     return combined results
```





Fork-Join Parallelism in Java

```
ForkJoinPool pool = new ForkJoinPool();
// array contains the integers to be summed
int[] array = new int[SIZE];

SumTask task = new SumTask(0, SIZE - 1, array);
int sum = pool.invoke(task);
```





Fork-Join Parallelism in Java

```
import java.util.concurrent.*;

public class SumTask extends RecursiveTask<Integer>
{
   static final int THRESHOLD = 1000;

   private int begin;
   private int end;
   private int[] array;

   public SumTask(int begin, int end, int[] array) {
      this.begin = begin;
      this.end = end;
      this.array = array;
   }

   protected Integer compute() {
      if (end - begin < THRESHOLD) {
         int sum = 0;
         for (int i = begin; i <= end; i++)
            sum += array[i];

         return sum;
      }
      else {
         int mid = (begin + end) / 2;

         SumTask leftTask = new SumTask(begin, mid, array);
         SumTask rightTask = new SumTask(mid + 1, end, array);

         leftTask.fork();
         rightTask.fork();

         return rightTask.join() + leftTask.join();
      }
   }
}
```



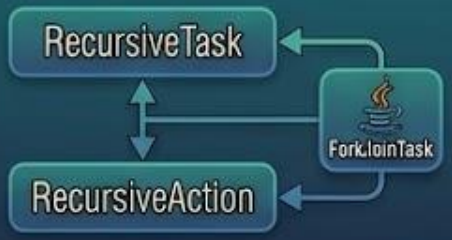
FORK-JOIN PARALLELISM IN JAVA

1. ForkJoinTask Base Class



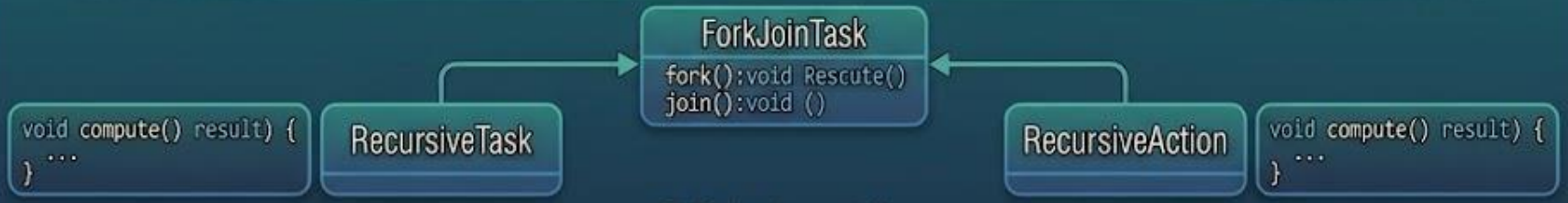
The ForkJoinTask is an abstract base class. It is the core of Java's Fork-Join framework, providing key operations for task management.

2. Key Task Extensions



RecursiveTask and RecursiveAction classes extend ForkJoinTask to common use cases.

Fork-Join Task Hierarchy & Behaviors



Returns a result (via the return value from the 'compute( )' method). Ideal for value-producing calculations.

Both implement the divide-and-conquer pattern by creating smaller sub-tasks.

Does not return a result. Perfect for tasks that perform 'work' (e.g., in-place array sorting) but produce no value.

KEY BENEFITS

RESULT HANDLING: Choose between value-producing or purely behavioral tasks.

ABSTRACT BASE: Provides standard operations for task reuse and extension.

RECURSIVE PATTERN: Natural fit for divide-and-conquer algorithm implementation.

FRAMEWORK USE: ForkJoinPool for efficient execution.

FORK-JOIN IN JAVA OVERVIEW

OPENMP (OPEN MULTI-PROCESSING)

TECHNICAL OVERVIEW

1. Compilers and API

C Compiler

C++ Compiler

FORTRAN Compiler

OpenMP API

Set of compiler directives and an API for C, C++, and FORTRAN.

2. Parallel Programming Support



Provides robust support for parallel programming in shared-memory environments.

Parallel Region Identification



Identifies parallel regions – blocks of code that can run in parallel. A master thread forks work to team threads.

Thread Management



Create as many threads as there are processor cores (e.g., matching physical or logical cores by default).

OPENMP PARALLEL REGION OVERVIEW

GRAND CENTRAL DISPATCH (GCD)

Apple Technology for macOS and iOS

1. Extensions and API

- C Extension — GCD API
- C++ Extension — GCD API
- Objective-C Extension — GCD API

Language Extensions, API, & Run-Time Library

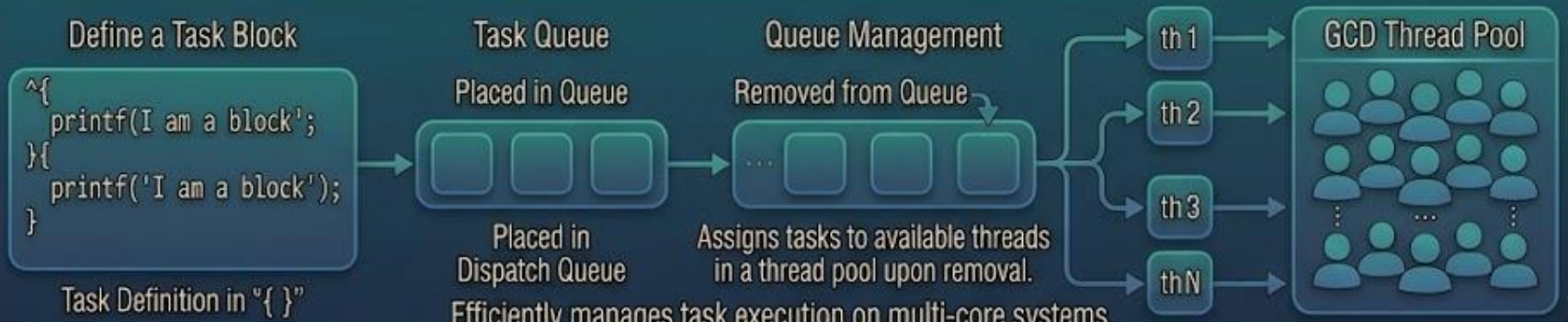
Extensions to C, C++ and Objective-C, comprehensive API, and background runtime library. Allows developers to identify parallel sections.

2. Automatic Thread Management



Manages most details of threading. Developers define tasks; GCD handles allocation and execution details.

GCD Block and Queue Process



Efficiently manages task execution on multi-core systems.

KEY CHARACTERISTICS

- **PARALLEL IDENTIFICATION:** Simplifies marking code sections
- **BLOCKS:** Reusable code units
- **QUEUES:** FIFO task sequencing
- **POOL:** Dynamic resource allocation
- **APPLE TECHNOLOGY:** Optimized for macOS & iOS

GRAND CENTRAL DISPATCH OVERVIEW

GCD GRAND CENTRAL DISPATCH (GCD)

Apple Technology for macOS and iOS

1. SERIAL DISPATCH QUEUES

FIFO



Blocks are removed in Strict FIFO Order.
A single queue per process, the Main Queue.
Additional queues can be created by programmers.



Assigned to the application's Main Thread.
All tasks are ordered.
Programmer Can Create additional serial queues.

2. CONCURRENT DISPATCH QUEUES



Tasks are removed in FIFO Order, but Several may be removed and run Simultaneously.



Allows for multiple tasks to execute in parallel, optimizing resource usage.

Four System-Wide Queues by Quality of Service



(1) USER INTERACTIVE
For tasks requiring immediate user feedback.



(2) USER INITIATED
For user-requested work that needs prompt results.



(3) UTILITY
For long-running tasks the user sees progress on (e.g., download).



(4) BACKGROUND
For tasks with no user awareness (e.g., indexing, backup).

DISPATCH QUEUE OVERVIEW



Grand Central Dispatch

- For the Swift language a task is defined as a closure – similar to a block, minus the caret
- Closures are submitted to the queue using the `dispatch_async()` function:

```
let queue = dispatch_get_global_queue
   (QOS_CLASS_USER_INITIATED, 0)

dispatch_async(queue,{ print("I am a closure.") })
```



INTEL® THREADING BUILDING BLOCKS (TBB)

Template Library for Parallel C++

TBB Overview



A comprehensive C++ template library for designing parallel programs. Simplifies complex parallelism.



Multi-threading

Serial Implementation (Simple for loop)

```
int main = () {
  for (int i=0; length; i++) {
    return {..};
}
```

Simple Serial For Loop



Strict Serial FIFO

Processor

Parallel Implementation (TBB parallel_for)

```
int state = {
  parallel_for(parallels__for Statement) {
    Tasks(for task length; i++) {
    if (int =0 < 188; i++) {
      return {...};
    }
  });
}
```

TBB parallel_for Statement



TBB Task Management

Dynamic Task Scheduler

Parallel Core Execution

INTEL TBB PARALLEL FOR OVERVIEW



Intel Threading Building Blocks (TBB)

- Template library for designing parallel C++ programs
- A serial version of a simple for loop

```
for (int i = 0; i < n; i++) {
   apply(v[i]);
}
```

- The same for loop written using TBB with **`parallel_for`** statement:

```
parallel_for (size_t(0), n, [=](size_t i) {apply(v[i]);});
```



THREADING ISSUES

1. SEMANTICS of fork() and exec() system calls

- When a thread calls **fork()**, does the new process duplicate all threads or only the calling one?
- Behavior varies (e.g., POSIX threads).
- **Exec()** typically replaces the entire process image, including all threads.

2. SIGNAL HANDLING

SYNCHRONOUS
(e.g., divide by zero, page fault)

- Delivered to the specific thread that caused it.

ASYNCHRONOUS
(e.g., Ctrl+C, timer)

- Can be delivered to a specific thread, all threads, or a dedicated thread.

3. THREAD CANCELLATION of target thread



ASYNCHRONOUS

- Terminated immediately (unsafe if resources held).

DEFERRED

- Checks a cancellation flag at specific points (safer).



4. THREAD-LOCAL STORAGE



- Each thread gets its own unique copy of static or global data.
- Useful for variables that should not be shared between threads.
- **Example:** unique identifiers, transaction IDs.

5. SCHEDULER ACTIVATIONS



- A mechanism for communication between user-level thread library and kernel.
- Kernel notifies library about events (e.g., thread blocking on I/O) via upcalls.
- Allows the library to make better scheduling decisions.

THREADING ISSUES: FORK() & EXEC()

1. MULTI-THREADED FORK() BEHAVIOR

UNIX VARIATION A: DUPLICATE ALL THREADS



UNIX VARIATION B:
DUPLICATE CALLING THREAD ONLY (POSIX)



****Note:** Some UNIXes support two versions of **fork()**.

2. EXEC() SYSTEM CALL BEHAVIOR



EXEC()

Standard EXEC() Functionality:

- Replaces the entire running process space.
- All existing threads are ***always*** terminated and replaced by the single thread of the new program.
- No variation among UNIX systems.



Signals and Signal Handling in UNIX Systems

UNIX Signals

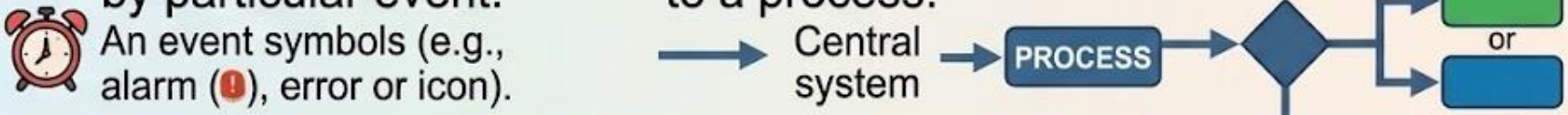


Used to notify a process that a particular **event** has **occurred**.



Signal Processing Workflow

1. Signal is generated by particular event.
   An event symbols (e.g., alarm (!), error or icon).
2. Signal is delivered to a process.
   → Central system → PROCESS
3. Signal is handled.



Types of Signal Handlers

default

- Kernel runs this when handling signal.
- Predefined behavior (e.g., terminate, ignore).

user-defined

- Can override default behavior.
- Custom code is executed.

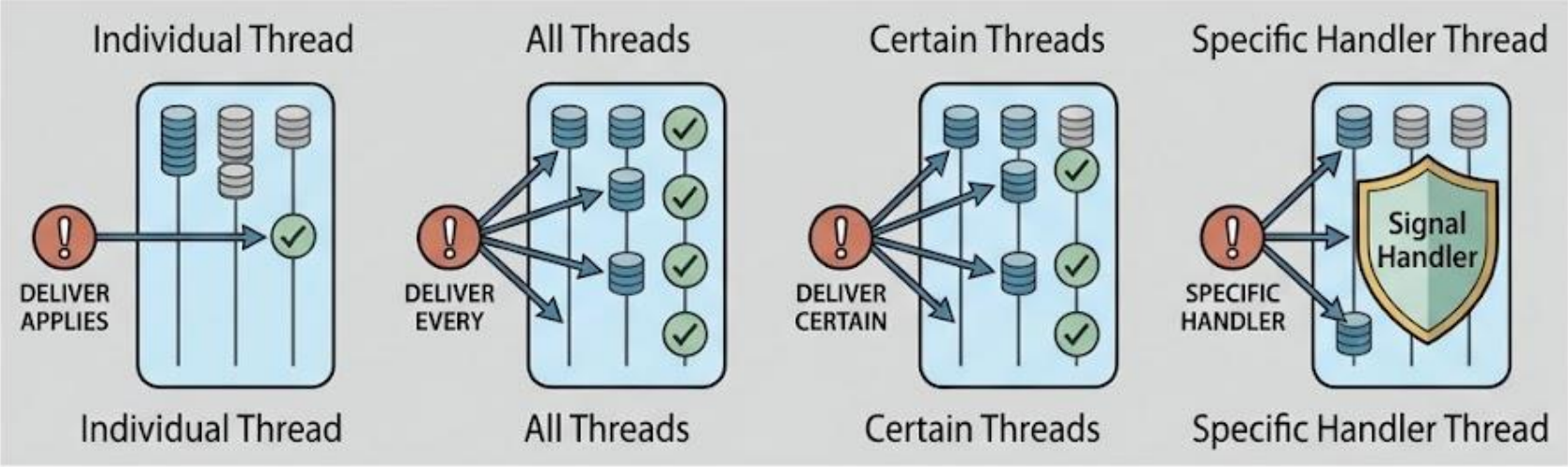
A **signal handler** is used to process signals

Key Points

- Every signal has **default handler** that kernel runs
- **User-defined signal handler** can override default
- For **single-threaded**, signal delivered to process

Signal Handling (Cont.)

- Where should a signal be delivered for multi-threaded?
  - Deliver the signal to the thread to which the signal applies
  - Deliver the signal to every thread in the process
  - Deliver the signal to certain threads in the process
  - Assign a specific thread to receive all signals for the process

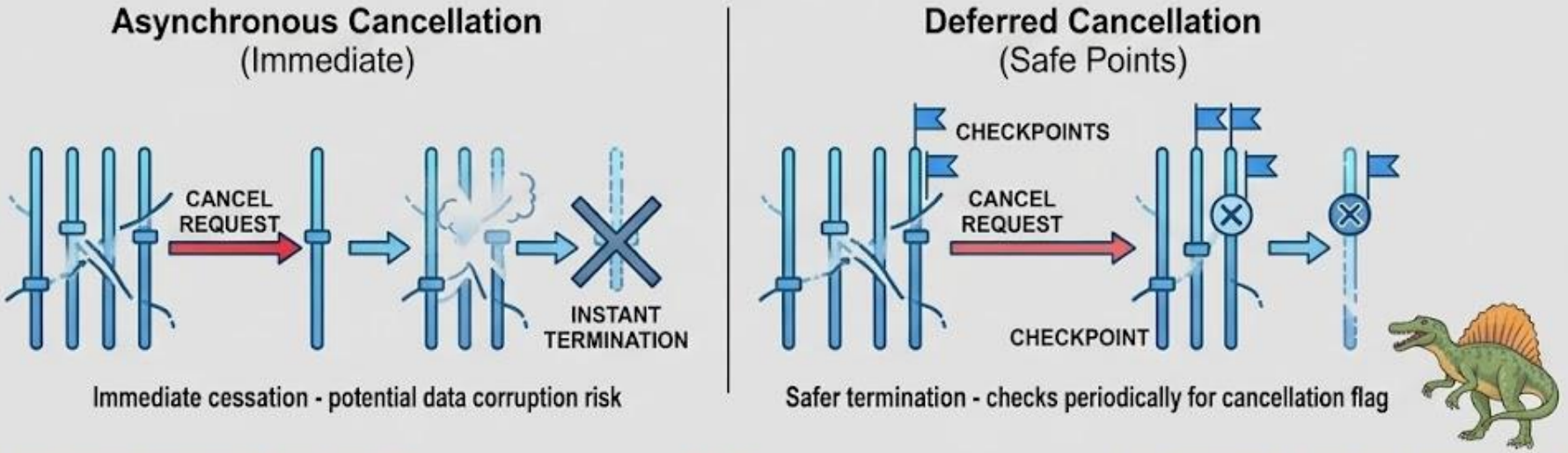




Thread Cancellation

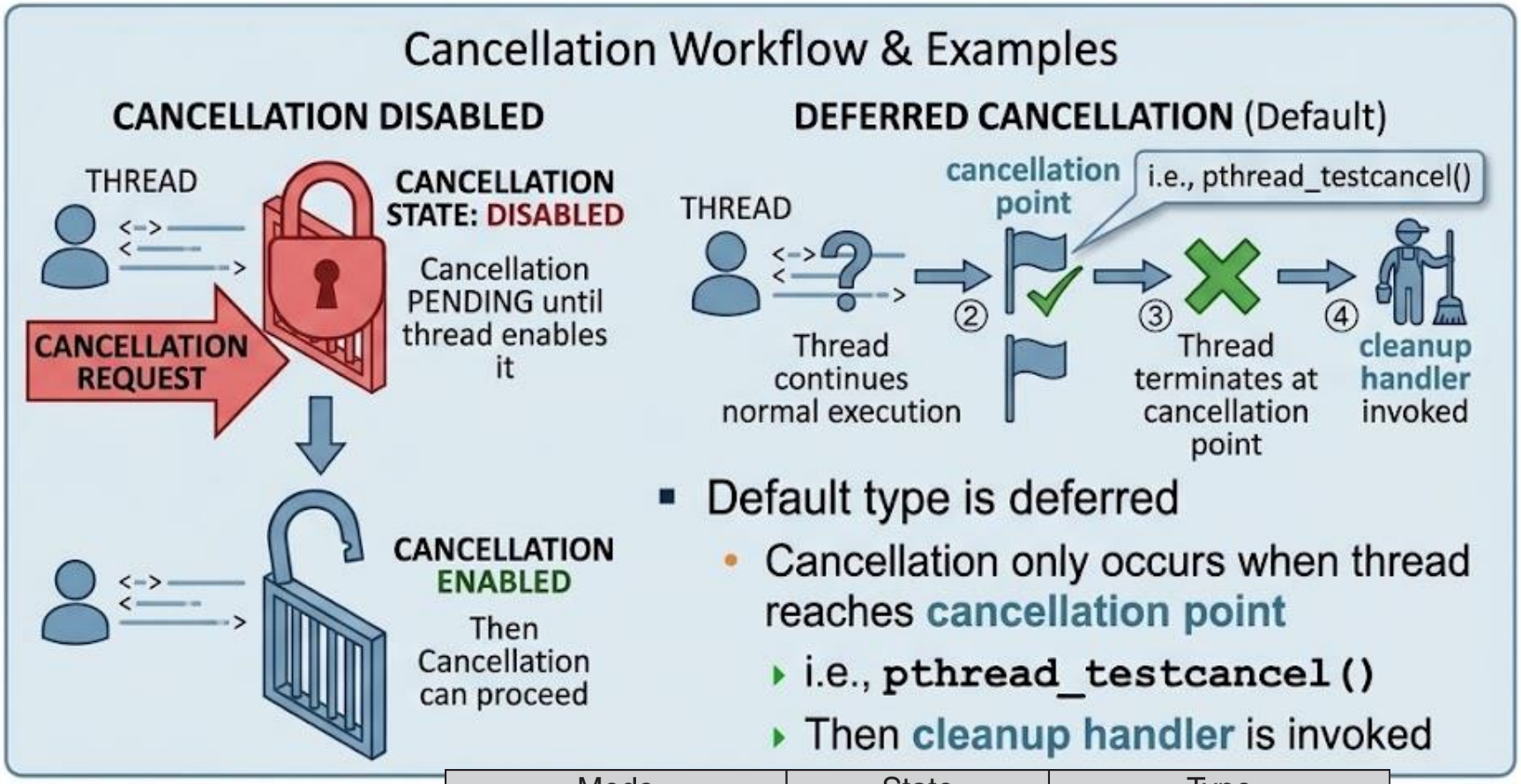
- Terminating a thread before it has finished
- Thread to be canceled is **target thread**
- Two general approaches:
  - **Asynchronous cancellation** terminates the target thread immediately
  - **Deferred cancellation** allows the target thread to periodically check if it should be cancelled
- Pthread code to create and cancel a thread:

Thread Cancellation Infographic



Thread Cancellation (Cont.)

- Invoking thread cancellation requests cancellation, but actual cancellation depends on thread state



- On Linux systems,

Mode	State	Type
Off	Disabled	–
Deferred	Enabled	Deferred
Asynchronous	Enabled	Asynchronous



Thread Cancellation in Java

- Deferred cancellation uses the **`interrupt()`** method, which sets the interrupted status of a thread.

```
Thread worker;

. . .

/* set the interruption status of the thread */
worker.interrupt()
```

- A thread can then check to see if it has been interrupted:

```
while (!Thread.currentThread().isInterrupted()) {
    . . .
}
```



THREAD-LOCAL STORAGE (TLS)

INTRODUCTION

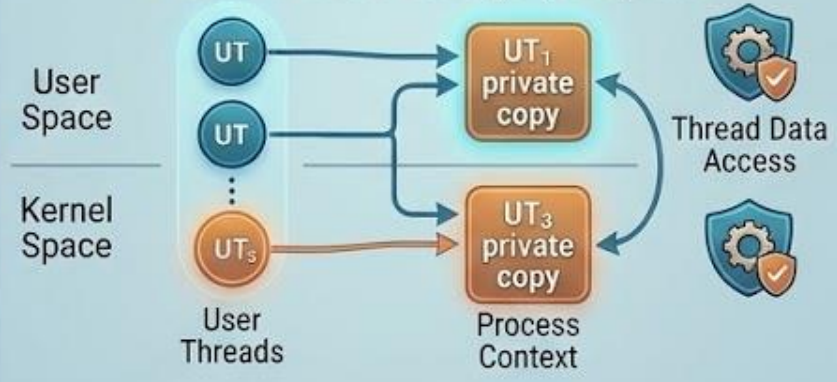
Thread-Local Storage (TLS) provides each thread in a multi: Prevents shared-state synchronization issues.

VISUALIZING THREAD-LOCAL STORAGE



- All threads access the same instance of global data. Requires complex locking (e.g., mutexes) to avoid race conditions.

Thread-Local Storage (TLS) Data



- Each thread accesses its own independent copy. Access is atomic from the thread's perspective, eliminating locking overhead for this data.

KEY CHARACTERISTICS OF TLS

- Thread-Unique Copies
- Persists per Thread Lifecycle
- Minimal Thread-Safety Issues (for local copies)
- Simplified Debugging for Thread-Specific State



PRIMARY BENEFITS

Performance: Reduced Synchronization Overheard

Simplicity: Fewer Shared State Pitfalls

Correctness: Inherently Thread-Safe for its use case

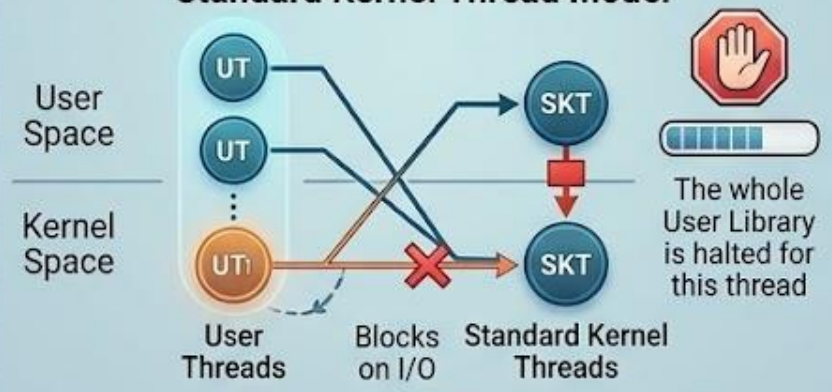
V1.0

SCHEDULER ACTIVATIONS: OPTIMIZING USER-LEVEL THREADS

INTRODUCTION

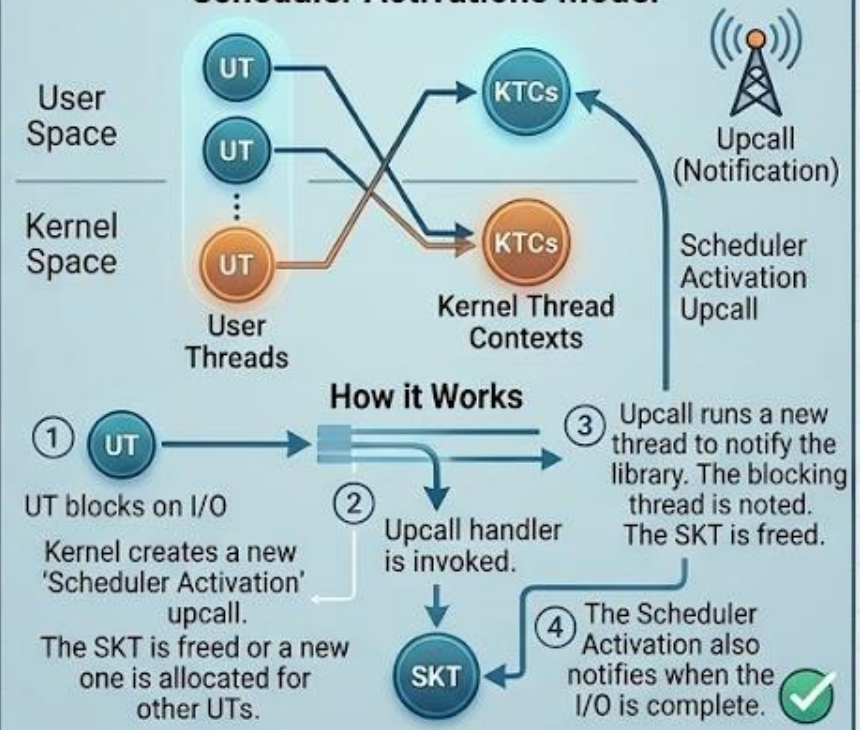
Bridge user-level thread libraries and kernel threads. Solve the blocking problem in M:M thread models.

Standard Kernel Thread Model



- **Blocking Problem:** If one user thread makes a blocking call, the kernel thread it is assigned to blocks, making it unavailable to other user threads.

Scheduler Activations Model



- Kernel provides upcalls and allocates kernel threads dynamically.

KEY CHARACTERISTICS

- Upcalls (Notifications)
- Kernel → User Thread Library Communication
- Dynamic Thread Allocation
- Reduced Scheduling Latency

BENEFITS

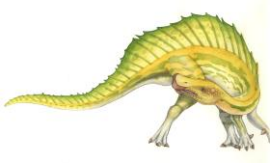
Enhanced Responsiveness | More Efficient CPU Utilization | Reduced Synchronization Overhead

V1.0



Operating System Examples

- Windows Threads
- Linux Threads

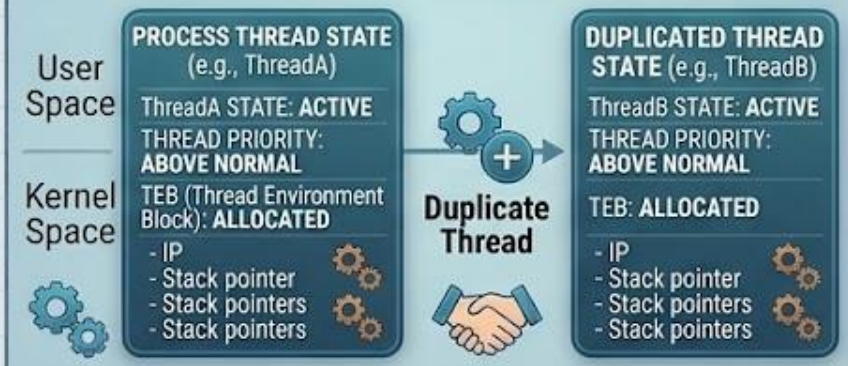


UNDERSTANDING WINDOWS THREADS

INTRODUCTION

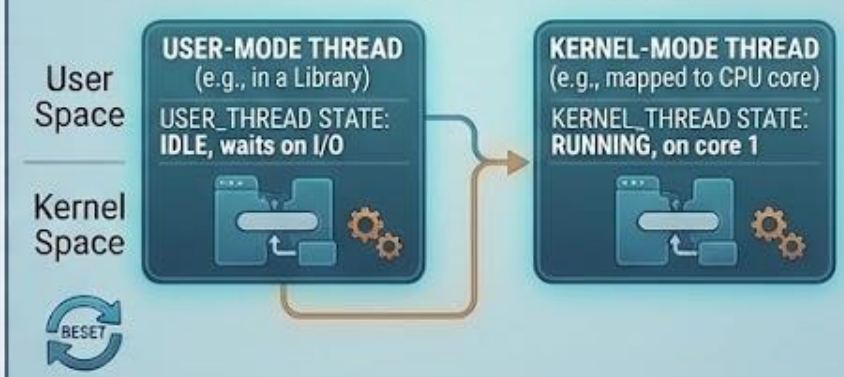
Windows Threads are fundamental units of execution within a process, managed directly by by the kernel.

THREAD STATE DUPLICATION & ATTRIBUTES



- **Dup() & Create() System Calls:** Thread contexts and attributes are duplicated and allocated. New TEB created. Full state copy (e.g., with CreateThread()).
- Pending thread events and contexts are generally not inherited.

KERNEL & USER MODE SCHEDULING



- **Scheduler Interaction:** User-mode threads (e.g., in M:N model) can be multiplexed. The Windows kernel manages actual Kernel-mode thread scheduling.
- Optimization (e.g., for user libraries, reduced synchronization).

KEY WINDOWS THREADING CONSIDERATIONS

- Unique Thread IDs & Attributes
- Kernel-Managed Priority System
- Thread Synchronization Objects (e.g., Mutexes, Events)
- Thread Environment Block (TEB) Structure

KERNEL & USER-MODE ADDLS SCHRINING

- Kernel-to-User communication actual Kernel-mode thread scheduling.
- Communication allows optimization (e.g., for user libraries, reduced synchronization).

KERNEL SCHEDULER

PRIMARY THREAD USES

Parallel Processing: Use multi-cores.

Background tasks, I/O wait.

Responsiveness: Main thread stays active.

V1.0



Windows Threads

- Windows API – primary API for Windows applications
- Implements the one-to-one mapping, kernel-level
- Each thread contains
  - A thread id
  - Register set representing state of processor
  - Separate user and kernel stacks for when thread runs in user mode or kernel mode
  - Private data storage area used by run-time libraries and dynamic link libraries (DLLs)
- The register set, stacks, and private storage area are known as the **context** of the thread





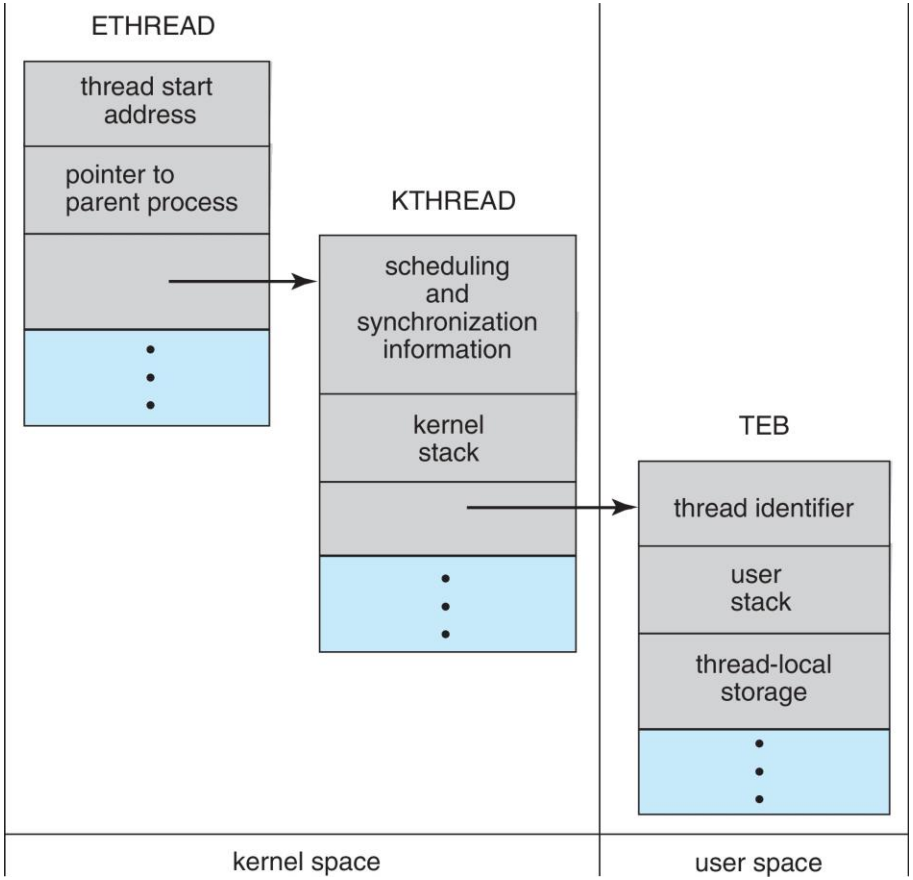
Windows Threads (Cont.)

- The primary data structures of a thread include:
  - ETHREAD (executive thread block) – includes pointer to process to which thread belongs and to KTHREAD, in kernel space
  - KTHREAD (kernel thread block) – scheduling and synchronization info, kernel-mode stack, pointer to TEB, in kernel space
  - TEB (thread environment block) – thread id, user-mode stack, thread-local storage, in user space





Windows Threads Data Structures







Linux Threads

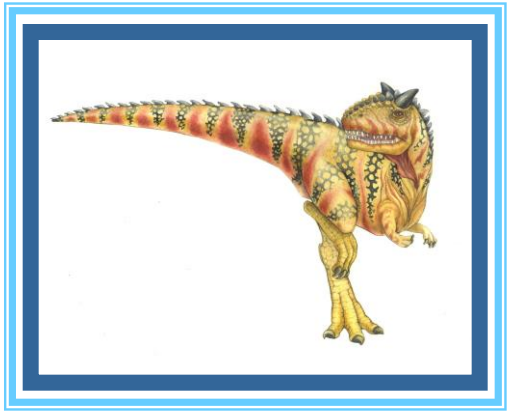
- Linux refers to them as ***tasks*** rather than ***threads***
- Thread creation is done through **`clone()`** system call
- **`clone()`** allows a child task to share the address space of the parent task (process)
  - Flags control behavior

flag	meaning
CLONE_FS	File-system information is shared.
CLONE_VM	The same memory space is shared.
CLONE_SIGHAND	Signal handlers are shared.
CLONE_FILES	The set of open files is shared.

- **`struct task_struct`** points to process data structures (shared or unique)



End of Chapter 4



Silberschatz, Galvin and Gagne ©2018